

ROS Utils

PyCRAM provides a number of utils to interact with the ROS network. The utils are:

- A TF Broadcaster
- A Joint State Publisher
- A Simulated Force Torque Sensor
- A Visualisation Marker Publisher
- A Robot State Updater

These site will go over all utils what they do and how to use them. All ROS utils presented here will publish continuously in a new thread. You can either stop the publishing by calling the `stop_publishing` or the thread will terminate automatically once the process is ended.

TF Broadcaster

The TF broadcaster broadcasts the transformations to every object and every link in the BulletWorld to the TF topic. This allows other ROS nodes to listen to the TF topic and transform their poses to the TF frames published in the transforms on the TF topic.

The broadcaster publishes transforms in a specific interval, this interval can be specified when creating the broadcaster.

```
from pycram.ros.tf_broadcaster import TFBroadcaster
```

```
broadcaster = TFBroadcaster()
```

The broadcaster allows to specify a projection namespace, the projection namespace will be prefixed before the TF frames. Furthermore, you can specify an odom frame and the interval at which the transforms will be published in seconds.

Joint State Publisher

The joint state publisher publishes the current joint positions for every joint of the currently loaded robot. Furthermore, allows the joint state publisher to specify a topic to publish as well as an interval at which the position should be published.

```
from pycram.ros.joint_state_publisher import JointStatePublisher
```

```
joint_publisher = JointStatePublisher("joint_states", 0.1)
```

Force Torque Sensor

PyBullet, the underlying simulation framework, allows to simulate a force torque sensor for a given joint. The results from this simulated force torque sensor are then published to a ROS topic of type `geometry_msgs/WrenchStamped`. Furthermore, allows the force torque sensor class to specify a topic as well as an interval in seconds.

```
from pycram.ros.force_torque_sensor import ForceTorqueSensor

ft_sensor = ForceTorqueSensor("l_wrist_roll_joint", "/pycram/fts", 0.1)
```

Visualisation Marker Publisher

The visualisation marker publisher sends an array of visualisation marker to a ROS topic which can then be rendered by RVIZ for example. The array consists of one Marker per link for each Object in the Bullet World, with each Object creating its own namespace. The visualisation marker publisher has to be instantiated once and then publishes constantly in the background.

```
from pycram.ros.viz_marker_publisher import VizMarkerPublisher

v = VizMarkerPublisher()
```

Robot State Updater

The robot state updater is intended for working with a real robot, since the robot in the bullet world should mimic the real robot there has to be a module which takes care of updating the state of the robot in the BulletWorld. The robot state updated hooks into the TF and joint_state topic to get the current position as well as the current joint configuration of the robot and applies it to the robot in the BulletWorld.

As all the other ROS utils the robot state updater only needs to be initialized and then takes care of himself.

To initialize the robot state updater a TF topic as well as a joint state topic have to be provided.

```
from pycram.ros.robot_state_updater import RobotStateUpdater

r = RobotStateUpdater("/tf", "/joint_states")
```

Adding a new robot to PyCRAM

To add a new robot to PyCRAM, you need two things:

- A Robot description

- Process Modules to control the robot

Robot Description

The robot description defines certain semantic properties of the robot that can not be extracted from the robot's URDF automatically. This includes kinematic chains which the robot can move (like the arms), descriptions of the end-effectors as well as descriptions of the cameras mounted on the robot.

An overview of the different components of the robot description as well as how these are created can be found in the following example:

[Robot Description example](#)

Process Modules

Process Modules are the components that actually control the robot. They are responsible for executing actions on the robot and are the only component of PyCRAM that interacts with the robot directly.

If you want to use the robot in simulation then you can use the provided default process modules which will suffice to control the robot in simulation. However, should the provided process modules not be sufficient for your use case, you can implement them yourself. For examples how the process modules are designed please have a look at the already implemented process modules in [pycram.process modules](#).

Designators

Designators are CRAMs and PyCRAMs way of representing actions, motions, objects and locations.

In general, PyCRAM-Designators consist of a description and a specified element. Descriptions describe sets of designators and designators are one thing in the described set. For example, such a description could describe an action where the robot moves to a location from where it can grasp an object. The specific location in this case is not relevant as long as the robot can reach the object.

The designator description will be resolved during runtime which results in a designator with specific parameter, the resulting designator can also be performed to let the robot perform the desired behaviour.

To stay on the example of an action designator which should move the robot to a location from where it can grasp an object, we will create a `NavigateAction` description with a list of possible poses.

```
poses = [Pose([1, 0, 0], [0, 0, 0, 1]), Pose([1.2, 0.2, 0], [0, 0, 1, 0])]
NavigateAction(target_locations=poses)
```

This is a description of an action which moves the robot to a pose in the environment. In this case the description describes a set of action designators which have a pose of the `poses` list as target location.

By resolving the description it will return a single designator with specific parameter. This can look like this:

```
designator = NavigateAction(target_locations=poses).resolve()
```

The resolver will check the available parameter and return a designator with appropriate parameter which can then be performed. The designator returned by the resolver will always be a sub-class of the description, in this way the designator can be assigned the type of action it performs. The resolved designator can also contain new parameter which might be relevant for performing the designator and will be inferred while resolving. A resolved action designator for a navigate action looks like this:

```
NavigateActionPerformable(robot_position=(Pose([0.0, 0.0, 0.0] [0.0, 0.0, 0.0, 1.0])), target_location=Pose([1, 0, 0], [0, 0, 0, 1]))
```

A visual representation of the whole idea of designator and designator descriptions can be seen in the following image.

There are four types of designators in PyCRAM:

- [`pycram.designators.action designator`](#)
- [`pycram.designators.object designator`](#)
- [`pycram.designators.location designator`](#)
- [`pycram.designators.motion designator`](#)

Object Designator

Object designators represent objects in (simulated) world. The description of object designators can take names and types that the object should match. The [`ground\(\)`](#) method returns an object with all its data attached that matches the description. The [`iter\(\)`](#) method iterates over all objects that match the description.

Contributing Object Designators

Object Designators should always be part of an object designator description. The general class structure is seen in [`ObjectDesignatorDescription`](#). New object description need to inherit from the general object description. If the object they ground to differs from the base object,

a [dataclass](#). should be created inside the new description. The dataclass is one element that matches the description. If ORM logging of the new objects is wanted a `to_sql()` and `insert()` method has to be implemented (see orm for more details).

Action Designator

Action designators describe complex actions that are executable for an agent. Action designators can be seen as higher level plans that include failure handling and parametrization. An action designator description always takes the parameter as a list of possible parameter, when resolving the description to a single designator one parameter out of the given list will be picked.

Motion Designator

Motion designators describe atomic actions that are executable for an agent. In contrast to action designators there is no failure handling or other action designators. Furthermore, the `perform()` method passes the resolved motion designator to the respective Process Module for execution on the robot.

Another difference to action designator is that motion designators only take a single parameter instead of a list, this parameter is also strictly typed.

Location Designator

Location designator describe a set of locations in regards to specific constrains. These constrains can be things like `reachable` or `visible`. The pose returned by a location designator is a single pose of the set defined by the constrains given to the location designator description.

Similar to object designator poses location designator also a [__iter__\(\)](#) method which can iterate over all possible solutions for this description.

Creating your own Designator

Creating your own designator is fairly easy, you only need to extend the base class of the respective description.

- [ActionDesignatorDescription](#)
- [ObjectDesignatorDescription](#)
- [LocationDesignatorDescription](#)
- [BaseMotion](#)

Afterwards you need to implement your own `ground` method which is the default resolver and for location and object designator it makes sense to also implement a `__iter__` method.

The `ground` and `__iter__` methods should return the designator sub-class so you also need to implement these with the parameter your designator needs.

The sub-class can already contain some parameters, this is usually the case if the parameter is the same for every designator of this type. For example, [Location](#) contains a `pose` parameter since every location designator contains a resolved pose.

For action and motion designator the sub-class is also the place where the `perform` method is written which contains the behaviour of the designator.

Knowledge

To be able to perform tasks in unknown environments a robot needs to have a way to reason and access knowledge about the world. This knowledge can be represented in different ways and formats. In this chapter we will discuss how PyCRAM access different kinds of knowledge and integrates them with action designators.

Concept

The concept of knowledge in PyCRAM is based on the idea that knowledge can be represented in different ways and provided from different sources which can be specialized for different tasks. These different sources of knowledge are implemented behind a common interface which provides a set of methods to query the knowledge. This architecture can be seen in the following image:

The methods provided by the knowledge sources, are called “properties” since they are used to reason about the properties of entities in the world. Properties can be combined to create more complex expressions which describe conditions that have to be true at the time an action designator is executed. Let’s look at an example explaining this:

```
GraspableProperty(ObjectDesignator(...))  
& ReachableProperty(Pose(...))
```

In this example, we have two properties, one that checks if an object is graspable and one that checks if a pose is reachable. The `&` operator is used to combine the two properties into a single property that checks if both conditions are true at the same time. This combined property stems from the `PickUpAction` where the object a robot wants to pick up has to be reachable for the robot as well as being able to fit into the end effector of the robot.

Since knowledge sources are usually specialized for a certain task, they do not need to be able to implement all methods of the interface. This leads to a lot of knowledge sources which all implement a subset of the methods, therefore no knowledge source can be used to answer all questions. To solve this problem, PyCRAM has a central interface for processing the combined properties and querying the knowledge sources called the “KnowledgeEngine”. The job of the KnowledgeEngine is to take a combined property and resolve the individual properties to the

available knowledge sources which implement the methods needed to answer the question. The resolved properties are then combined in the same way as the input property and evaluated.

This image shows the process of resolving the properties through the knowledge engine:

Knowledge Sources

Knowledge does not have a unified form or representation, it can be available as an SQL database, a Knowledge Graph, a simple JSON file, etc. To be able to handle a multitude of different representations of knowledge, PyCRAM uses the concept of Knowledge Sources. A Knowledge Source is a class that implements a set of methods to access knowledge. Therefore, PyCRAM does not care how the knowledge is accessed or where it is from as long as the Knowledge Source implements the abstract methods.

The methods that a Knowledge Source must implement are some basic methods to manage connecting to the knowledge itself and more importantly, methods to query the knowledge. Which methods are provided by each knowledge source decides each knowledge source on its own by using the respective property as a mix-in of the knowledge source. The properties currently available and which a knowledge source can implement are:

- *GraspableProperty*: Checks if an object is graspable
- *ReachableProperty*: Checks if a pose is reachable
- *SpaceIsFreeProperty*: Checks if a space is free for the robot to move to
- *GripperIsFreeProperty*: Checks if the gripper is free to grasp an object
- *VisibleProperty*: Checks if an object is visible

If you want to learn more about the implementation of a Knowledge Source, you can look at the following example:

[Knowledge Source example](#)

Knowledge Engine

The Knowledge Engine is the central component in PyCRAM to reason about the world. It takes a combined property and resolves the individual properties to the available knowledge sources which implement the methods needed to answer the question.

While the properties are resolved they also infer parameters which are needed to execute the action but may not be defined in the action designator description. For example, the `PickUpAction` needs an arm to pick up an object with, however, the arm does not need to be defined in the action designator and can be inferred from the properties and the state of the world.

After the properties are resolved, evaluated and the parameters are inferred, the Knowledge Engine grounds the action in the belief state and tests if the found solution is valid and can achieve the goal. If the solution is valid, the Knowledge Engine returns the solution and the action designator is performed.

Knowledge and Reasoning

The knowledge engine is able to infer parameters of a designator description from the context given by the properties attached to its parameters. Since the properties are defined for the parameters of a designator description they add semantic information to the designator description parameters. The knowledge engine is able to utilize this information to infer the value of a parameter from the context.

Inference is done very similar to the normal reasoning process where the property function of the designator description is first resolved and then evaluated. The difference is that we now not only look at the result (if the properties are satisfied or not) but also at the possible parameter solutions that are generated while reasoning.

We start again by taking the properties of the designator description and resolve them. This was already explained in the Knowledge documentation.

We then evaluate the properties and generate the possible parameter solutions.

As can be seen in the picture above the major part of inferring missing parameter is done by the Knowledge Source with the semantic context provided by the properties. The magic consists now of matching the inferred parameter from the Knowledge Sources with the parameter of the designator description.

Matching of inferred parameters

The parameters that are inferred by the Knowledge Source during reasoning need to be matched to the designator to be usable in the execution of a designator. Matching of the inferred parameters is kinda ambiguous since the parameters are provided by the Knowledge Source as a dictionary with a name and the value. Therefore the name given in the dictionary might not match the designator.

To solve the issue of aligning the inferred parameters from the Knowledge Source with the parameters of the designator we employ two methods. The first is to match the names in the dictionary with the names of the parameters of the designator. This is most reliable when the

return from the Knowledge Source tries to adhere to the conventions of the designator description. The second method is to match the type of the inferred parameter with the type annotations in the designator. While this seems like the more reliable method, it could happen that a designator has multiple parameters of the same type. In this case the matching might not yield the correct result, since the first found parameter of the designator is matched with the parameter of the Knowledge Source.

Costmaps

Costmaps are a way to describe positions in a defined area around a pose with respect to certain constraints. For example, there is a costmap which contains every position from which a certain object is visible.

In PyCRAM these costmaps are used to dynamically generate poses for certain criteria like visibility, reachability or occupancy. So if you, for example, want to find a position from where the robot can see a certain object you would generate a costmap for the visibility and one for occupancy. These costmaps can then be merged with one another and result in a costmap which contains every position from which the object is visible and where the robot can stand.

Currently there are four types of costmaps implemented:

- Occupancy Costmap
- visibility Costmap
- Gaussian Costmap
- Semantic Costmap

Occupancy Costmap

Occupancy costmaps represent all positions that don't have any objects positioned above them. Meaning the robot can stand there without colliding with anything. The occupancy costmap can be generated in two ways, either by using a map provided by the ROS map_server or by directly generating it from the BulletWorld. Additionally user can specify a value by which obstacles should be inflated creating a boundary around obstacles to avoid colliding with them. Below you can see the constructor for the Occupancy Costmap if it is generated from the ROS map_server.

```
from pycram.costmaps import OccupancyCostmap
```

```
occupancy = OccupancyCostmap(0.2, True)
```

In this case it is enough to specify the value by which obstacles should be inflated as well as the boolean which says that this costmap should be generated from the ROS map_server. The value is given in metre meaning 0.2 translates to 20 centimetre. Since the costmap that generated in

this way is rather large and usually encompasses the whole environment the robot is operating in there is a method that can cut out a smaller map from the whole occupancy map. This is done to get a local costmap that is centered around the point of interest. This is also necessary since the merging of costmaps requires that all given costmaps have the same size.

```
from pycram.datastructures.pose import Pose

local_occupancy = occupancy._create_sub_map(Pose([1, 0.3, 0]), 200)
```

The parameter given to the method are the new origin around which the local costmap is centered and the size the costmap should have.

In the case where the costmap is generated from the BulletWorld the constructor gets more parameter, specifying the exact dimensions of the costmap.

```
from pycram.costmaps import OccupancyCostmap
from pycram.datastructures.pose import Pose

occupancy = OccupancyCostmap(distance_to_obstacle=0.2,
                              from_ros=False,
                              size=200,
                              resolution=0.02,
                              origin=Pose([1, 0.3, 0]))
```

The given parameter are:

- The inflation radius
- The the costmap should not be created from Ros
- The size
- The resolution
- The origin

The inflation radius was already explained above while the boolean just specifies if the cosmap should be created from the ROS map_server or the BulletWorld. The size specifies the height and width of the costmap, costmaps are usually created as a square. The resolution determines how many metre a pixel in the costmap represents in the real world. The origin is the pose, around which the costmap is centered.

You can see an image of the final Occupancy costmap with an inflation radius of 0.2 m below.

Visibility Costmap

Visibility costmaps show the visibility for a specific position in a restricted area. This means every position from which the robot can see the position given as the map origin.

The visibility costmap is created by taking depth images from the origin position of the costmap and then checking which positions are occluded by other objects. Essentially, this specifies how far you can look from the object in all direction. Afterwards, a 2D representation is created from these depth images.

The constructor for the visibility costmap gets the following parameter

```
from pycram.costmaps import VisibilityCostmap
from pycram.datastructures.pose import Pose

visibility = VisibilityCostmap(min_height=1.27,
                              max_height=1.6,
                              size=200,
                              resolution=0.02,
                              origin=Pose([1, 0.3, 0]))
```

The parameter for the constructor are:

- Minimal height of the camera
- Maximal height of the camera
- Size of the costmap
- Resolution of the costmap
- Origin of the costmap

The minimal and maximal height of the camera specify the height of the camera from the ground. This is in case the camera can move, either if the torso of the robot moves or the camera itself is movable. Since the movement of the camera gives the robot more opportunities to see the pose this is taken into account with these parameters. If the camera on the robot can not change its height, just set both parameter to the height of the camera.

Size of the costmap is the height and width of the resulting costmap. Resolution is how many metre a pixel in the costmap represents in the real world. Origin is the position the costmap is centered around as well as the position for which the visibility is calculated.

A simple visibility costmap with two objects can be seen below.

Gaussian Costmap

A gaussian costmap is essentially a 2D gauss distribution with its peak at the centre of the the costmap. Gaussian costmaps are, for example, used to approximate reachability of objects. The idea being that to reach an object the robot has to be relatively close to the object to reach it.

Since all other objects use just 0 or 1 to represent if an entry in the costmap is valid according to their respective constraint Gaussian Costmaps add some variance to highlight a certain point.

This is especially useful when keeping in mind that sampling from the costmap is based on the maximum likelihood, meaning the cells with the highest value will be sampled first.

```
from pycram.costmaps import GaussianCostmap
from pycram.datastructures.pose import Pose

gauss = GaussianCostmap(mean=200,
                        sigma=15,
                        resolution=0.02,
                        origin=Pose([1, 0.3, 0]))
```

The parameter given to the gaussian costmap are:

- Mean of gaussian distribution
- Sigma of the gaussian distribution
- Resolution of the costmap
- Origin of the costmap

The mean is the mean value of the gaussian distribution as well as the size of the resulting costmap. Sigma is the sigma value of the gaussian distribution which is contained in the costmap. Resolution and origin are the same as for other costmaps.

A plot of the gaussian costmap can be seen below. This is a matplotlib plot of the costmap to better show the distribution.

Semantic Costmap

Semantic costmaps allow to specify a costmap based on a semantic description of the environment. This means that the costmap is created given an object and a link in the belief state. The costmap is created by taking the bounding box of the given link and creating a distribution above it.

The semantic costmap is meant to be used to create poses for symbolic descriptions of the environment. For example, if the robot should place an object on the table but no specific pose is given, the semantic costmap can be used to create a costmap that contains all possible positions on the table. A sample of this costmap can then be used to parametrize the plan.

A semantic costmap is created by specifying the object and the link in the belief state. Optionally the resolution of the resulting costmap can also be specified.

```
from pycram.costmaps import GaussianCostmap

semantic = SemanticCostmap(object=apartment,
                          link="table_area_main",
                          resolution=0.02)
```

The image below show a semantic costmap created for a table.

Algebraic Semantic Costmap

A algebraic semantic costmap is a special for of a semantic costmap. It is created using [Random Event](#). This allows to slice the created costmap to specify the area of interest. For example, the costmap is created above the whole table but we only want to place an object on the lower right area or the outer perimeter of the table. This can be done as follows:

```
from pycram.costmaps import GaussianCostmap

semantic = AlgebraicSemanticCostmap(object=apartment,
                                     link="table_area_main")
semantic.valid_area &= semantic.right()
semantic.valid_area &= semantic.bottom()
```

The result of this operation can be seen in the image below.

Visualization of Costmaps

Costmaps can be visualized in the BulletWorld, for this every costmap contains the method “visualize” which can be directly called for the costmap.

```
visibility.visualize()
```

To make the visualization of a costmap disappear from the BulletWorld you can simply call the “close_visualization” method for the same costmap.

```
visibility.close_visualization()
```

The images for the occupancy and visibility costmap show the visualisation in the BulletWorld with this method. Debugging Visualization.

For a more comprehensive visualization of the costmap the is the method “plot_grid” in the costmap.py file. This creates a matplotlib plot of the 2D numpy array which represents the costmap. The “plot_grid” method can be called as follows:

```
from pycram.costmaps import VisibilityCostmap
from pycram.datastructures.pose import Pose

visibility = VisibilityCostmap(1.27, 1.6, 200, 0.02, Pose([1, 0.3, 0]))
plot_grid(visibility.map)
```

The image for the gaussian costmap shows the plot of matplotlib. Merging of Costmaps.

It is possible to merge different costmaps to create a costmap that contains positions that adhere to more than one constraint. For example, if you merge a visibility and occupancy costmap you get a costmap that contains positions where the robot can stand and see a specific point.

To be able to merge different costmaps there are a few restrictions that you have to follow. The restrictions are:

- The costmaps must have the same size
- The costmaps must have the same origin
- The costmaps must have the same resolution

These restrictions make it much easier to merge costmaps and also reduce the probability of errors occurring in the resulting costmap. Since all for all costmaps these parameter can be set when creating them, it shouldn't pose a problem to match these parameter for all created costmaps, making them able to be merged.

The one exception for this is the occupancy costmap when generated from the ROS map_server, in this case all parameter are already decided by the map_server. However, for this case there is the “_create_sub_map” method which creates a local costmap from the bigger occupancy costmap. The only thing that can not be fixed with this is the resolution of the occupancy map, which stays the same for the local costmap. But then again, since when creating the other costmaps a resolution can be specified the resolution given for the other costmaps can be adjusted to the one of the occupancy costmap.

Troubleshooting

This page contains the most common errors that could happen when using PyCRAM and how to resolve them.

Ros Init Exception

`ROSInitException: time is not initialized. Have you called init_node()`

This exception usually occurs when trying to load an Object into the BulletWorld or when creating a Pose. The reason for this exception is that the ROS node of PyCRAM could not be initialized, this is usually the case when the PyCRAM launch file was not launched.

To solve this problem you just have to launch the `ik_and_description` file of PyCRAM. This can be done via the following command.

```
roslaunch pycram ik_and_description
```

Robot Description not Loaded

In PyCRAM a lot of things are base on the robot description that is currently loaded and in turn the robot description that will be loaded depends on the robot description that is on the ROS parameter server at the time PyCRAM is started.

If you get an error that looks similar to the following exception the most likely case is that the robot description was not loaded.

```
763 with open(self.path) as f:
764     self.urdf_object = URDF.from_xml_string(f.read())
--> 765     if self.urdf_object.name == robot_description.name and not
BulletWorld.robot:
766         BulletWorld.robot = self
768 self.links[self.urdf_object.get_root()] = -1

AttributeError: 'NoneType' object has no attribute 'name'
```

There could be a few reasons for this behaviour:

- **The name of the URDF on the parameter server and the name in the PyCRAM robot description are different**
 - The PyCRAM robot description is matched by the name of the robot in the URDF
 - You can check the assignment of the PyCRAM robot description in this function `update_robot_description()` (there is a link to the source code)
- You only started a roscore
- You did not start the `ik_and_description` launch file

Stop Iteration

Stop iterations usually happen when you try to resolve a designator for which there is no solution or if you iterate over a designator and reached the end of all possible solutions.

When you try to resolve a designator which has no solution the error will look something like this.

```
753 def ground(self) -> Union[Object, bool]:
754     """
755     Return the first object from the bullet world that fits
the description.
756
757     :return: A resolved object designator
758     """
--> 759     return next(iter(self))
```

StopIteration:

If you encounter such an error the most likely reason is that you put the wrong arguments into your DesignatorDescription. The best solution is to double check the input arguments of the DesignatorDescription.

BulletWorld crashes after loading an URDF

It can happen that the BulletWorld crashes when loading a URDF. This happens when loading a URDF with more than 127 links. This is a limitation of PyBullet which can not deal with objects with more links.

Only real solution here is to either delete links such that you have at max 127 links or split up the URDF and load the split parts individually.

Error when performing Actions or Motions

```
30 def perform(self):
31     pm_manager = ProcessModuleManager.get_manager()
--> 32     return pm_manager.navigate().execute(self)
```

AttributeError: 'NoneType' object has no attribute 'navigate'

If you get an error like this when trying to perform an action or motion designator, then you did not specify how the designator should be executed. You can specify how the designator should be performed by using the `simulated_robot` or `real_robot` environments. This is also explained in the [Action Designator Example](#).

```
with simulated_robot:
    NavigateAction([Pose()]).resolve().perform()
```

JntArray index out of Range

This error originates in the `kdl_ik_service` which is responsible for calculating ik solutions and usually looks like this: .. code-block:: python

```
ServiceException: service [/kdl_ik_service/get_ik] responded with an error: b'error
processing request: JntArray index out of range'
```

This error can have two origins.

- **The URDF on the parameter server and the URDF of the robot for which an ik solution should be found are not identical**
 - For example: The URDF on the parameter server is from the PR2 and a solution for the HSR should be found
- The amount of names of joints and the amount of joint positions in the `robot_state` part of the request are not equal.

General Remarks

Jupyter Notebooks & PyCharm

If you have the situation that you activate your venv that has rospy installed and launched PyCharm from the console you'll run into trouble using jupyter notebooks. PyCharm allows you to correctly set up the jupyter server, however the notebook will always use your `/usr/bin/python` as interpreter and you can't select the correct one in the drop down menu.

To fix this issue one has to execute

```
python -m ipykernel install --user --name <kernel_name> --display-name  
"<Name_to_display>"  
, eg.
```

```
python -m ipykernel install --user --name pycram --display-name "pycram"
```

in your terminal. `--name` is the name of your virtual environment and `--display-name` is the name that will display in the drop down menu of jupyter. After that, select the correct Python interpreter kernel (`pycram`) and everything should work now. Refer [here](#) for details.

Dirty Terminals

If your terminal gets polluted by PyBullet complaining about incomplete URDF descriptions, you need to first fix your URDF files by inserting the missing tags and second delete the *resources/cached* folder.

Bullet World

The BulletWorld is the internal simulation of PyCRAM. You can simulate different actions and reason about the outcome of different actions.

It is possible to spawn objects and robots into the BulletWorld, these objects can come from URDF, OBJ or STL files.

A BulletWorld can be created by simply creating an object of the BulletWorld class.

```
from pycram.worlds.bullet_world import BulletWorld  
from pycram.world_concepts.world_object import Object  
from pycram.datastructures.enums import ObjectType, WorldMode  
from pycram.datastructures.pose import Pose  
from pycrap.ontologies import Milk, Cereal, Robot, Kitchen  
  
world = BulletWorld(mode=WorldMode.DIRECT)
```

```

milk = Object("milk", Milk, "milk.stl")
pr2 = Object("pr2", Robot, "pr2.urdf")
cereal = Object("cereal", Cereal, "breakfast_cereal.stl", pose=Pose([1.4,
1, 0.95]))

```

The BulletWorld allows to render images from arbitrary positions. In the following example we render images with the camera at the position [0.3, 0, 1] and pointing towards [1, 0, 1], so we are looking upwards along the x-axis.

The renderer returns 3 different kinds of images which are also shown on the left side of the BulletWorld window. (If these windows are missing, click the BulletWorld window to focus it, and press “g”) These images are:

- An RGB image which shows everything like it is rendered in the BulletWorld window, just from another perspective.
- A depth image which consists of distance values from the camera towards the objects in the field of view.
- A segmentation mask image which segments the image into the different objects displayed. The segmentation is done by assigning every pixel the unique id of the object that is displayed there.

```

world.get_images_for_target(Pose([1, 0, 1], [0, 0, 0, 1]), Pose([0.3, 0,
1], [0, 0, 0, 1]))

```

Objects

Everything that is located inside the BulletWorld is an Object. Objects can be created from URDF, OBJ or STL files. Since everything is of type Object a robot might share the same methods as a milk (with some limitations).

Signature: Object:

- Name
- Type
- Filename or Filepath

Optional:

- Position
- Orientation
- World
- Color
- Ignore Cached Files

Objects provide methods to change the position and rotation, change the color, attach other objects, set the state of joints if the objects has any or get the position and orientation of a link.

These methods are the same for every Object, however since some Objects may not have joints or more than one link methods related to these will not work.

```
milk.set_position(Pose([1, 0, 0]))
```

Since everything inside the BulletWorld is an Object, even a complex environment Object like the kitchen can be spawned in the same way as the milk.

```
kitchen = Object("kitchen", Kitchen, "kitchen.urdf")
```

Costmaps

Costmaps are a way to get positions with respect to certain criterias. The currently available costmaps are:

- [OccupancyCostmap](#)
- [VisibilityCostmap](#)
- [SemanticCostmap](#)
- [GaussianCostmap](#)

It is also possible to merge multiple costmaps to combine different criteria.

Visibility Costmaps

Visibility costmaps determine every position, around a target position, from which the target is visible. Visibility Costmaps are able to work with cameras that are movable in height for example, if the robot has a movable torso.

```
import pycram.costmaps as cm
```

```
v = cm.VisibilityCostmap(1.27, 1.60, size=300, resolution=0.02,  
origin=Pose([0, 0, 0.1], [0, 0, 0, 1]))  
v.visualize()  
v.close_visualization()
```

Occupancy Costmap

Is valid for every position where the robot can be placed without colliding with an object.

```
o = cm.OccupancyCostmap(0.2, from_ros=False, size=300, resolution=0.02,  
origin=Pose([0, 0, 0.1], [0, 0, 0, 1]))  
s = cm.SemanticCostmap(kitchen, "kitchen_island_surface", resolution=0.02)
```

```
g = cm.GaussianCostmap(200, 15, resolution=0.02)
```

You can visualize the costmap in the BulletWorld to get an impression what information is actually contained in the costmap. With this you could also check if the costmap was created correctly. Visualization can be done via the ‘visualize’ method of each costmap.

```
o.visualize()  
o.close_visualization()
```

It is also possible to combine two costmap, this will result in a new costmap with the same size which contains the information of both previous costmaps. Combination is done by checking for each position in the two costmaps if they are zero, in this case to same position in the new

costmap will also be zero in any other case the new position will be the normalized product of the two combined costmaps.

```
ov = o + v
ov.visualize()
ov.close_visualization()
```

Bullet World Reasoning

Allows for geometric reasoning in the BulletWorld. At the moment the following types of reasoning are supported:

- [stable\(\)](#)
- [contact\(\)](#)
- [visible\(\)](#)
- [occluding\(\)](#)
- [reachable\(\)](#)
- [blocking\(\)](#)
- [supporting\(\)](#)

We start with testing for visibility

```
from pycram.world_reasoning import visible

milk.set_position(Pose([1, 0, 1]))
visible = visible(milk, pr2.get_link_pose("wide_stereo_optical_frame"))
print(f"Milk visible: {visible}")
from pycram.world_reasoning import contact
from pycram.datastructures.world import World

milk.set_position(Pose([1, 0, 0.05]))

plane = World.current_world.objects[0]
contact = contact(milk, plane)
print(f"Milk is in contact with the floor: {contact}")
from pycram.world_reasoning import reachable

milk.set_position(Pose([0.6, -0.5, 0.7]))

reachable = reachable(milk, pr2, "r_gripper_tool_frame")
print(f"Milk is reachable for the PR2: {reachable}")
```

Designators

Designators are symbolic descriptions of Actions, Motions, Objects or Locations. In PyCRAM the different types of designators are represented by a class which takes a description, the description then tells the designator what to do.

For example, let's look at a Motion Designator to move the robot to a specific location.

Motion Designators

When using a Motion Designator you need to specify which Process Module needs to be used, either the Process Module for the real or the simulated robot. A Process Module is the interface between a real or simulated robot, and PyCRAM designators. By exchanging the Process Module, one can quickly change the robot the plan is executed on, allowing PyCRAM plans to be re-used across multiple robot platforms. This can be done either with a decorator which can be added to a function and then every designator executed within this function, will be executed on the specified robot. The other possibility is a “with” scope which wraps a code piece.

These two ways can also be combined, you could write a function which should be executed on the real robot and the function contains a “with” scope which executes something on the simulated robot for reasoning purposes.

```
from pycram.designators.motion_designator import *
from pycram.process_module import simulated_robot, with_simulated_robot

description = MoveMotion(target=Pose([1, 0, 0], [0, 0, 0, 1]))

with simulated_robot:
    description.perform()

from pycram.process_module import with_simulated_robot

@with_simulated_robot
def move():
    MoveMotion(target=Pose([0, 0, 0], [0, 0, 0, 1])).perform()

move()
```

Other implemented Motion Designator descriptions are:

- Accessing
- Move TCP
- Looking
- Move Gripper
- Detecting
- Move Arm Joint
- World State Detecting

Object Designators

An Object Designator represents objects. These objects could either be from the BulletWorld or the real world. Object Designators are used, for example, by the PickupAction to know which object should be picked up.

```
from pycram.designators.object_designator import *

milk_desig = BelieveObject(names=["milk"])
milk_desig.resolve()
```

Location Designator

Location Designator can create a position in cartesian space from a symbolic description

```
from pycram.designators.object_designator import *

milk_desig = BelieveObject(names=["milk"])
milk_desig.resolve()
```

Location Designators

Location Designators can create a position in cartesian space from a symbolic description.

```
from pycram.designators.location_designator import *
from pycram.designators.object_designator import *

robot_desig = BelieveObject(types=[Robot]).resolve()
milk_desig = BelieveObject(names=["milk"]).resolve()
location_desig = CostmapLocation(target=milk_desig,
visible_for=robot_desig)

print(f"Resolved: {location_desig.resolve()}")
print()

for pose in location_desig:
    print(pose)
```

Action Designator

Action Designators are used to describe high-level actions. Action Designators are usually composed of other Designators to describe the high-level action in detail.

```
from pycram.designators.action_designator import *
from pycram.datastructures.enums import Arms
```

```
with simulated_robot:
    ParkArmsAction([Arms.BOTH]).resolve().perform()
```

Making a simple plan

To get familiar with the PyCRAM Framework we will write a simple pick and place plan. This plan will let the robot grasp a cereal box from the kitchen counter and place it on the kitchen island. This is a simple pick and place plan.

```
from pycram.datastructures.enums import Grasp, TorsoState

cereal_desig = ObjectDesignatorDescription(names=["cereal"])
kitchen_desig = ObjectDesignatorDescription(names=["kitchen"])
robot_desig = ObjectDesignatorDescription(names=["pr2"]).resolve()
with simulated_robot:
    ParkArmsAction([Arms.BOTH]).resolve().perform()

    MoveTorsoAction([TorsoState.HIGH]).resolve().perform()

    pickup_pose = CostmapLocation(target=cereal_desig.resolve(),
    reachable_for=robot_desig).resolve()
    pickup_arm = pickup_pose.reachable_arms[0]

    NavigateAction(target_locations=[pickup_pose.pose]).resolve().perform()

    PickUpAction(object_designator_description=cereal_desig,
    arms=[pickup_arm], grasps=[Grasp.FRONT]).resolve().perform()

    ParkArmsAction([Arms.BOTH]).resolve().perform()

    place_island = SemanticCostmapLocation("kitchen_island_surface",
    kitchen_desig.resolve(),

    cereal_desig.resolve()).resolve()

    place_stand = CostmapLocation(place_island.pose,
    reachable_for=robot_desig, reachable_arm=pickup_arm).resolve()

    NavigateAction(target_locations=[place_stand.pose]).resolve().perform()

    PlaceAction(cereal_desig, target_locations=[place_island.pose],
    arms=[pickup_arm]).resolve().perform()

    ParkArmsAction([Arms.BOTH]).resolve().perform()
```

Task Trees

Task trees are a hierarchical representation of all Actions involved in a plan. The Task tree can later be used to inspect and restructure the execution order of Actions in the plan.

```
import pycram.tasktree
import anytree

tt = pycram.tasktree.task_tree
print(anytree.RenderTree(tt))
from anytree.dotexport import RenderTreeGraph, DotExporter

RenderTreeGraph(tt).to_picture("tree.png")
```

ORM

```
import sqlalchemy.orm
import pycram.orm.base
import pycram.orm.action_designator

# set description of what we are doing
pycram.orm.base.ProcessMetaData().description = "Tutorial for getting
familiar with the ORM."

engine = sqlalchemy.create_engine("sqlite+pysqlite:///memory:",
echo=False)
session = sqlalchemy.orm.Session(bind=engine)
pycram.orm.base.Base.metadata.create_all(engine)
session.commit()

tt.insert(session)
from sqlalchemy import select

navigations =
session.scalars(select(pycram.orm.action_designator.NavigateAction)).all()
print(*navigations, sep="\n")
navigations = (session.scalars(
    select(pycram.orm.action_designator.NavigateAction,
pycram.orm.base.Position, pycram.orm.base.Quaternion).
    join(pycram.orm.action_designator.NavigateAction.pose).
    join(pycram.orm.base.Pose.position).
    join(pycram.orm.base.Pose.orientation)).all())
```

```
print(*navigations, sep="\n")
```

The world can also be closed with the ‘exit’ method

```
world.exit()
```

Bullet World

This Notebook will show you the basics of working with the PyCRAM BulletWorld.

First we need to import and create a BulletWorld.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import ObjectType, WorldMode
from pycrap.ontologies import Milk, Cereal, Robot
```

```
world = BulletWorld(mode=WorldMode.DIRECT)
```

This new window is the BulletWorld, PyCRAMs internal physics simulation. You can use the mouse to move the camera around:

- Press the left mouse button to rotate the camera
- Press the right mouse button to move the camera
- Press the middle mouse button (scroll wheel) and move the mouse up or down to zoom

At the moment the BulletWorld only contains a floor, this is spawned by default when creating the BulletWorld. Furthermore, the gravity is set to 9.8 m/s², which is the same gravitation as the one on earth.

To spawn new things in the BulletWorld we need to import the Object class and create an instance of it.

```
from pycram.world_concepts.world_object import Object
```

```
milk = Object("milk", Milk, "milk.stl", pose=Pose([0, 0, 1]))
```

As you can see this spawns a milk floating in the air. What we did here was create a new Object which has the name "milk" as well as the type [MILK](#), is spawned from the file "milk.stl" and is at the position [0, 0, 1].

The type of an Object can either be from the enum ObjectType or a string. However, it is recommended to use the enum since this would make for a more consistent naming of

types which makes it easier to work with types. But since the types of the enum might not fit your case you can also use strings.

The first three of these parameters are required while the position is optional. As you can see it was sufficient to only specify the filename for PyCRAM to spawn the milk mesh. When only providing a filename, PyCRAM will search in its resource directory for a matching file and use it.

For a complete list of all parameters that can be used to crate an Object please check the documentation.

Since the Object is spawned, we can now interact with it. First we want to move it around and change its orientation

```
milk.set_position(Pose([1, 1, 1]))
milk.set_orientation(Pose(orientation=[1, 0, 0, 1]))
milk.set_pose(Pose([0, 0, 1], [0, 0, 0, 1]))
```

In the same sense as setting the position or orientation, you can also get the position and orientation.

```
print(f"Position: \n{milk.get_position()}")

print(f"Orientation: \n{milk.get_orientation()}")

print(f"Pose: \n{milk.get_pose()}")
```

Attachments

You can attach Objects to each other simply by calling the attach method on one of them and providing the other as parameter. Since attachments are bi-directional it doesn't matter on which Object you call the method.

First we need another Object

```
cereal = Object("cereal", Cereal, "breakfast_cereal.stl", pose=Pose([1, 0, 1]))
milk.attach(cereal)
```

Now since they are attached to each other, if we move one of them the other will move in conjunction.

```
milk.set_position(Pose([1, 1, 1]))
```

In the same way the Object can also be detached, just call the detach method on one of the two attached Objects.

```
cereal.detach(milk)
```

Links and Joints

Objects spawned from mesh files do not have links or joints, but if you spawn things from a URDF like a robot they will have a lot of links and joints. Every Object has two dictionaries as attributes, namely `links` and `joints` which contain every link or joint as key and a unique id, used by PyBullet, as value.

We will see this at the example of the PR2:

```
pr2 = Object("pr2", Robot, "pr2.urdf")
print(pr2.links)
```

For links there are similar methods available as for the pose. However, you can only **get** the position and orientation of a link.

```
print(f"Position: \n{pr2.get_link_position('torso_lift_link')}")

print(f"Orientation: \n{pr2.get_link_orientation('torso_lift_link')}")

print(f"Pose: \n{pr2.get_link_pose('torso_lift_link')}")
```

Methods available for joints are:

- [get joint position\(\)](#)
- [set joint position\(\)](#)
- [get joint limits\(\)](#)

We will see how these methods work at the example of the `torso_lift_joint`:

```
print(f"Joint limits: {pr2.get_joint_limits('torso_lift_joint')}")

print(f"Current Joint state:
{pr2.get_joint_position('torso_lift_joint')}")

pr2.set_joint_position("torso_lift_joint", 0.2)

print(f"New Joint state: {pr2.get_joint_position('torso_lift_joint')}")
```

Misc Methods

There are a few methods that don't fit any category but could be helpful anyway. The first two are `get_color()` and `set_color()`, as the name implies they can be used to get or set the color for specific links or the whole Object.

```
print(f"Pr2 forearm color: {pr2.get_link_color('r_forearm_link')}")
pr2.set_link_color("r_forearm_link", [1, 0, 0])
```

Lastly, there is [get axis aligned bounding box\(\)](#), AABB stands for Axis Aligned Bounding Box. This method returns two points in world coordinates which span a rectangle representing the AABB.

```
pr2.get_axis_aligned_bounding_box()
```

To close the BulletWorld again please use the [`exit\(\)`](#) method since it will also terminate threads running in the background

```
world.exit()
```

Plan Language

The PyCRAM plan language is a way to structure the execution of your plan. In generally the plan language allows to execute designators either sequential or in parallel. Furthermore, exceptions that occur during execution of a plan with the plan language do not interrupt the execution instead they are caught and saved to a dictionary for later analysis. All language expressions return a State, this can either be SUCCEEDED or FAILED.

There are 4 language expressions:

Expression	Name	Description
+	Sequential	Executes the designators one after another, if one of the designators raises an exception execution is aborted and the state FAILED will be returned.
-	Try In Order	Executes the designators one after another, if one designator raises an exception the exception is caught and saved but the execution is not interrupted and the other designators are executed. Returns the state SUCCEEDED if at least one designator can be executed without exception.
*	Repeat	Repeat the previous language expression a number of time. Has to be used with a language expression and an integer.
	Parallel	Executes all designators in parallel. For each designator there will be a new thread created. If one designator is executed in this thread. If one of the designators raises an exception the return state will be FAILED.
^	Try All	Executes all designators in parallel with a designated thread for each designator. Returns SUCCEEDED if at least one designator can be executed without an exception.
>>	Monitor	Monitors the execution of the attached language expression, will interrupt the execution as a given condition is fulfilled.

The Sequential expression is the only one which aborts the execution once an error is raised.

When using the plan language a tree structure of the plan is created where the language expressions are nodes and designators are leafs. This tree uses AnyTree (like the task tree) and can be rendered with the anytree Renderer.

Sequential

This language expression allows to execute designators one after another, if one of the designators raises an exception the execution will be aborted and the state FAILED will be returned.

We will start with a simple example that uses an action designator for moving the robot and parking its arms.

```
import time

from pycram.designators.action_designator import *
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import Arms

navigate = NavigateAction([Pose([1, 1, 0])])
park = ParkArmsAction([Arms.BOTH])

plan = navigate + park
```

With this simple plan created we can inspect it and render the created tree structure.

```
from anytree import RenderTree

print(RenderTree(plan))
```

As you can see there is the root node which is the language expression and then there are the leafs which are the designators. When executing this plan the Sequential node will try to execute the NavigateAction and if that is finished without any error the ParkArmsAction will be executed.

The plan can be executed by wrapping it inside a with `simulated_robot` environment and calling `perform` on the plan.

If you are performing a plan with a simulated robot, you need a `BulletWorld`.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import ObjectType
from pycrap.ontologies import Robot

world = BulletWorld()
pr2 = Object("pr2", Robot, "pr2.urdf")

from pycram.process_module import simulated_robot

world.reset_world()

with simulated_robot:
    plan.perform()
```

Try In Order

Try in order is similar to Sequential, it also executes all designators one after another but the key difference is that an exception in one of the designators does not terminate the whole execution. Furthermore, the state FAILED will only be returned if all designator executions raise an error.

Besides the described difference in behaviour this language expression can be used in the same way as Sequential.

```
from pycram.designators.action_designator import *
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot

world.reset_world()

navigate = NavigateAction([Pose([1, 1, 0])])
park = ParkArmsAction([Arms.BOTH])

plan = navigate - park

with simulated_robot:
    plan.perform()
```

Parallel

Parallel executes all designator at once in dedicated threads. The execution of other designators is not aborted when a exception is raised, this is the case since threads can not be killed from the outside and this would also cause unforeseen problems. The state returned will be SUCCEEDED if all designators could be executed without an exception raised in any other case FAILED will be returned.

Since executing designators in parallel can get chaotic especially with complex actions like PickUp or Transport. For this reason not all action designators can be used in parallel and try all expressions. The list of action designator that cannot be used in language expressions can be seen in [parallel blocklist](#).

Designators that cannot be used in parallel and try all:

- PickupAction
- PlaceAction
- OpenAction
- CloseAction
- TransportAction

Using the parallel expressions works like Sequential and TryInOrder.

```

from pycram.designators.action_designator import *
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot

world.reset_world()

navigate = NavigateAction([Pose([1, 1, 0])])
park = ParkArmsAction([Arms.BOTH])

plan = navigate | park

with simulated_robot:
    plan.perform()

```

Try All

TryAll is to Parallel what TryInOrder is to Sequential, meaning TryAll will also execute all designators in parallel but will return SUCCEEDED if at least one designator is executed without raising an exception.

TryAll can be used like any other language expression.

```

from pycram.designators.action_designator import *
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot

world.reset_world()

navigate = NavigateAction([Pose([1, 1, 0])])
park = ParkArmsAction([Arms.BOTH])

plan = navigate ^ park

with simulated_robot:
    plan.perform()

```

Combination of Expressions

You can also combine different language expressions to further structure your plans. If you combine sequential and parallel expression please keep in mind that sequential expressions bind stronger than parallel ones. For example:

```

navigate | park + move_torso

```

In this case ‘park’ and ‘move_torso’ would form a Sequential expression and ‘navigate’ would form a Parallel expression with Sequential. You can try this yourself in the following cell.

```
from pycram.designators.action_designator import *
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot

world.reset_world()

navigate = NavigateAction([Pose([1, 1, 0])])
park = ParkArmsAction([Arms.BOTH])
move_torso = MoveTorsoAction([TorsoState.HIGH])

plan = navigate | park + move_torso

with simulated_robot:
    plan.perform()
```

Code Objects

You can not only use designators in the plan language but also python code. For this there is the [Code](#) object which takes a callable and the arguments for this callable. This allows you to execute arbitrary code in a plan.

The callable that is used in the [Code](#) object can either be a lambda expression or, for more complex code, a function. If you use a function you can provide parameters as keyword-arguments.

```
from pycram.designators.action_designator import *
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot
from pycram.language import Code

def code_test(param):
    print("-" * 20)
    print(param)

park = ParkArmsAction([Arms.BOTH])
code = Code(lambda: print("This is from the code object"))
code_func = Code(code_test, {"param": "Code function"})

plan = park | code | code_func

with simulated_robot:
```

```
plan.perform()
```

Exception Handling

If an exception is raised during the execution of a designator when it is used in a language expression the exception will be caught and saved to a dictionary. In general all designators in a language expression are executed regardless of exceptions raised, the only exception from this is the Sequential expression which stops after it encountered an exception.

The language will only catch exceptions that are of type **PlanFailure** meaning errors that are defined in `plan_failures.py` in PyCRAM. This also means normal Python errors, such as `KeyError`, will interrupt the execution of your designators.

We will see how exceptions are handled at a simple example.

```
from pycram.designators.action_designator import *
from pycram.process_module import simulated_robot
from pycram.language import Code
from pycram.failures import PlanFailure

def code_test():
    raise PlanFailure

navigate = NavigateAction([Pose([1, 1, 0])])
code_func = Code(code_test)

plan = navigate | code_func

with simulated_robot:
    plan.perform()

print(plan.exceptions)
```

Repeat

Repeat simply repeats a language expression a number of times. As all other language expressions Repeat captures exceptions that occur during execution and saves them to the dictionary in the root of the plan.

Since Repeat uses the `*` operator you should keep in mind that it will be evaluated before any other operator, so use parentheses to ensure the correct structure of your plan.

You can see an example of how to use Repeat below.

```

from pycram.designators.action_designator import *
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import TorsoState

move_torso_up = MoveTorsoAction([TorsoState.HIGH])
move_torso_down = MoveTorsoAction([TorsoState.LOW])

plan = (move_torso_up + move_torso_down) * 5

with simulated_robot:
    plan.perform()

```

Monitor

Monitor allows to monitor the execution of a language expression and interrupt it as soon as a given condition is fulfilled. The condition can either be a Callable which returns a boolean or a Fluent. When executed the Monitor will create a separate thread which will check if the condition is satisfied with a frequency of 10 Hz. If the condition is satisfied the execution of the language expression will be interrupted.

For the example on how Monitors work we will use the previous example with the robot moving up and down. We will use a Monitor to interrupt the execution after 2 seconds instead of executing the whole plan 5 times.

```

from pycram.designators.action_designator import *
from pycram.process_module import simulated_robot
from pycram.language import Monitor
import time
from pycram.datastructures.enums import TorsoState

move_torso_up = MoveTorsoAction([TorsoState.HIGH])
move_torso_down = MoveTorsoAction([TorsoState.LOW])

def monitor_func():
    time.sleep(2)
    return True

plan = (move_torso_up + move_torso_down) * 5 >> Monitor(monitor_func)

with simulated_robot:
    plan.perform()

```

If you are finished with this example you can close the world with the cell below.

```

world.exit()

```

Local Transformer

The local transformer is used to handle transforms between different frames in PyCRAM. This is useful when you want to transform a pose from one frame to another, for example, from the map frame to the frame of an object. This example will introduce the Local Transformer and how to use it to transform poses between frames.

Setting up the Environment

This step involves importing the required modules and initializing key components for our tasks.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.pose import Transform, Pose
from pycram.local_transformer import LocalTransformer
from pycram.datastructures.enums import WorldMode
```

Initializing the World

Every robot simulation requires a world where it can interact. This world serves as the playground where the robot performs tasks. Let's start by creating this world.

Since the local transformer can only transform between frames of objects which are in the world, we need to create a world first.

```
# Create an instance of the BulletWorld
world = BulletWorld(WorldMode.DIRECT)
```

Adding Objects to the World

For our robot to perform meaningful tasks, we need to populate its world with objects. In this section, we'll add a variety of objects, from a simple floor plane to kitchen setups and items like milk and bowls. These objects will be used in subsequent tasks, to provide the frames to which we will transform poses.

```
from pycram.worlds.bullet_world import Object
from pycram.datastructures.enums import ObjectType
from pycrap.ontologies import Kitchen, Milk, Bowl

kitchen = Object("kitchen", Kitchen, "kitchen.urdf")
milk = Object("milk", Milk, "milk.stl", pose=Pose([0.9, 1, 0.95]))
bowl = Object("bowl", Bowl, "bowl.stl", pose=Pose([1.6, 1, 0.90]))
```

Creating a Local Transformer

The local transformer is implemented as a singleton, meaning regardless of how much and where an instance is created it will always be the same instance. This is done since the local transformer collects all transformations between frames and would there always be a new instance, all transformations would need to be re-collected.

```
from pycram.local_transformer import LocalTransformer

local_transformer = LocalTransformer()
print(local_transformer)

new_local_transformer = LocalTransformer()
print(new_local_transformer)
```

Transformations with LocalTransformer

Now that we have our world set up, let's perform some transformations. We'll use the LocalTransformer to transform poses relative to our objects.

```
from pycram.local_transformer import LocalTransformer

l = LocalTransformer()
test_pose = Pose([1, 1, 1], [0, 0, 0, 1], "map")

transformed_pose = l.transform_to_object_frame(test_pose, milk)
print(transformed_pose)

print("-----")
new_pose = l.transform_pose(transformed_pose, "map")
print(new_pose)
```

In the above code, we first transformed a pose to the object frame of the milk object, and then we transformed it back to the map frame. This demonstrates how we can easily manipulate poses relative to objects in our environment. You can also transform poses relative to other poses. by using the transform_pose method. Further you can set a Transform.

```
from pycram.datastructures.pose import Transform

l.set_transform(Transform([1, 1, 1], [0, 0, 0, 1], "map", "test_frame"),
"example")
p = Pose()

transformed_pose = l.transform_pose(p, "test_frame")
```

Transformation frames

Links of an Object are represented by the Object frame_id + '/' + link name. Since link names need to be unique for an URDF this is no problem.

These frames need to be used in whenever you are transforming something with the local transformer. To get the base frame of an Object, meaning the frame name without any link, there is the attribute tf_frame and for the frame of a link there is a method which returns the frame name of a link given the link name.

```
print(milk.tf_frame)
```

```
print(kitchen.get_link_tf_frame("kitchen_island_surface"))
```

You can use the cell below to exit the simulation.

```
world.exit()
```

TaskTree Tutorial

In this tutorial we will walk through the capabilities of task trees in pycram.

First we have to import the necessary functionality from pycram.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.robot_description import RobotDescription
import pycram.tasktree
from pycram.datastructures.enums import Arms, ObjectType
from pycram.designators.action_designator import *
from pycram.designators.location_designator import *
from pycram.process_module import simulated_robot
from pycram.designators.object_designator import *
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import ObjectType, WorldMode, TorsoState
import anytree
import pycram.failures
```

Next we will create a bullet world with a PR2 in a kitchen containing milk and cereal.

```
from pycrap.ontologies import Milk, Cereal, Robot, Kitchen
world = BulletWorld(WorldMode.DIRECT)
pr2 = Object("pr2", Robot, "pr2.urdf")
kitchen = Object("kitchen", Kitchen, "kitchen.urdf")
milk = Object("milk", Milk, "milk.stl", pose=Pose([1.3, 1, 0.9]))
cereal = Object("cereal", Cereal, "breakfast_cereal.stl", pose=Pose([1.3,
0.7, 0.95]))
milk_desig = ObjectDesignatorDescription(names=["milk"])
cereal_desig = ObjectDesignatorDescription(names=["cereal"])
robot_desig = ObjectDesignatorDescription(names=["pr2"]).resolve()
```

```
kitchen_desig = ObjectDesignatorDescription(names=["kitchen"])
```

Finally, we create a plan where the robot parks his arms, walks to the kitchen counter and picks the cereal and places it on the table. Then we execute the plan.

```
@pycram.tasktree.with_tree
def plan():
    with simulated_robot:
        ParkArmsActionPerformable(Arms.BOTH).perform()
        MoveTorsoAction([TorsoState.MID]).resolve().perform()
        pickup_pose = CostmapLocation(target=cereal_desig.resolve(),
        reachable_for=robot_desig).resolve()
        pickup_arm = pickup_pose.reachable_arms[0]

        NavigateAction(target_locations=[pickup_pose.pose]).resolve().perform()
        PickUpAction(object_designator_description=cereal_desig,
        arms=[pickup_arm], grasps=[Grasp.FRONT]).resolve().perform()
        ParkArmsAction([Arms.BOTH]).resolve().perform()

        place_island = SemanticCostmapLocation("kitchen_island_surface",
        kitchen_desig.resolve(),

        cereal_desig.resolve()).resolve()

        place_stand = CostmapLocation(place_island.pose,
        reachable_for=robot_desig, reachable_arm=pickup_arm).resolve()

        NavigateAction(target_locations=[place_stand.pose]).resolve().perform()

        PlaceAction(cereal_desig, target_locations=[place_island.pose],
        arms=[pickup_arm]).resolve().perform()

        ParkArmsAction([Arms.BOTH]).resolve().perform()

        ParkArmsActionPerformable(Arms.BOTH).perform()

    plan()
```

Now we get the task tree from its module and render it. Rendering can be done with any render method described in the anytree package. We will use ascii rendering here for ease of displaying.

```
tt = pycram.tasktree.task_tree
print(anytree.RenderTree(tt.root))
```

As we see every task in the plan got recorded correctly. It is noticeable that the tree begins with a NoOperation node. This is done because several, not connected, plans that get executed after each other should still appear in the task tree. Hence, a NoOperation node is the root of any tree. If we re-execute the plan we would see them appear in the same tree even though they are not connected.

```
world.reset_world()
plan()
```

```
print(anytree.RenderTree(tt.root))
```

Projecting a plan in a new environment with its own task tree that only exists while the projected plan is running can be done with the `with` keyword. When this is done, both the bullet world and task tree are saved and new, freshly reset objects are available. At the end of a `with` block the old state is restored. The root for such things is then called `simulation()`.

```
with pycram.tasktree.SimulatedTaskTree() as stt:
    print(anytree.RenderTree(pycram.tasktree.task_tree.root))
print(anytree.RenderTree(pycram.tasktree.task_tree.root))
```

Task tree can be manipulated with ordinary `anytree` manipulation. If we for example want to discard the second plan, we would write

```
tt.root.children = (tt.root.children[0],)
print(anytree.RenderTree(tt.root, style=anytree.render.AsciiStyle()))
```

We can now re-execute this (modified) plan by executing the leaf in pre-ordering iteration using the `anytree` functionality. This will not append the re-execution to the task tree.

```
world.reset_world()
with simulated_robot:
    [node.action.perform() for node in tt.root.leaves]
print(anytree.RenderTree(pycram.tasktree.task_tree,
style=anytree.render.AsciiStyle()))
```

Nodes in the task tree contain additional information about the status and time of a task.

```
print(pycram.tasktree.task_tree.children[0])
```

The task tree can also be reset to an empty one by invoking

```
pycram.tasktree.task_tree.reset_tree()
print(anytree.RenderTree(pycram.tasktree.task_tree,
style=anytree.render.AsciiStyle()))
```

If a plan fails using the `PlanFailure` exception, the plan will not stop. Instead, the error will be logged and saved in the task tree as a failed subtask. First let's create a simple failing plan and execute it.

```
@pycram.tasktree.with_tree
def failing_plan():
    raise pycram.failures.PlanFailure("Oopsie!")

try:
    failing_plan()
except pycram.failures.PlanFailure as e:
    print(e)
```

We can now investigate the nodes of the tree, and we will see that the tree indeed contains a failed task.

```
print(anytree.RenderTree(pycram.tasktree.task_tree,
style=anytree.render.AsciiStyle()))
print(pycram.tasktree.task_tree.children[0])
world.exit()
```

Improving Actions using Probabilities

In this tutorial we will look at probabilistic specifications of actions and especially at an advance plan to pick up objects. After this tutorial you will know:

- Why are probabilities useful for robotics
- How to use probabilistic models to specify actions
- How to use probabilistic machine learning to improve actions

Let's start by importing all the necessary modules.

```
import numpy as np
import os
import random

import pandas as pd
import sqlalchemy.orm

import plotly
from probabilistic_model.probabilistic_circuit.nx.probabilistic_circuit
import ProbabilisticCircuit

plotly.offline.init_notebook_mode()
import plotly.graph_objects as go
import tqdm

from probabilistic_model.learning.jpt.jpt import JPT
from probabilistic_model.learning.jpt.variables import
infer_variables_from_dataframe
from random_events.product_algebra import Event, SimpleEvent

import pycram.orm.base
from pycram.designators.action_designator import
MoveTorsoActionPerformable
from pycram.failures import PlanFailure
from pycram.designators.object_designator import
ObjectDesignatorDescription
from pycram.worlds.bullet_world import BulletWorld
```

```

from pycram.world_concepts.world_object import Object
from pycram.robot_descriptions import robot_description
from pycram.datastructures.enums import ObjectType, WorldMode
from pycram.datastructures.pose import Pose
from pycram.ros_utils.viz_marker_publisher import VizMarkerPublisher
from pycram.process_module import ProcessModule, simulated_robot
from
pycram.designators.specialized_designators.probabilistic.probabilistic_act
ion import MoveAndPickUp, Arms, Grasp
from pycram.tasktree import task_tree, TaskTree

ProcessModule.execution_delay = False
np.random.seed(69)
random.seed(69)

```

Next, we connect to a database where we can store and load robot experiences.

```

pycorm_uri = "robot_enjoyer:I_love_robots_123@neem-2.informatik.uni-
bremen.de:3306/pycram_ci"
pycorm_uri = "mysql+pymysql://" + pycorm_uri
engine = sqlalchemy.create_engine(pycorm_uri)
session = sqlalchemy.orm.sessionmaker(bind=engine) ()
# pycram.orm.base.Base.metadata.create_all(engine)

```

Now we construct an empty world with just a floating milk, where we can learn about PickUp actions.

```

from pycrap.ontologies import Robot, Milk

world = BulletWorld(WorldMode.DIRECT)
print(world.prospection_world)
robot = Object("pr2", Robot, "pr2.urdf")
milk = Object("milk", Milk, "milk.stl", pose=Pose([1.3, 1, 0.9]))
viz_marker_publisher = VizMarkerPublisher()
viz_marker_publisher = VizMarkerPublisher()
milk_description = ObjectDesignatorDescription(types=[Milk]).ground()

```

Next, we create a default, probabilistic model that describes how to pick up objects. We visualize the default policy. The default policy tries to pick up the object by standing close to it, but not too close.

```

fpa = MoveAndPickUp(milk_description, arms=[Arms.LEFT, Arms.RIGHT],
                    grasps=[Grasp.FRONT.value, Grasp.LEFT.value,
Grasp.RIGHT.value, Grasp.TOP.value])
print(world.current_world)
p_xy = fpa.policy.marginal([fpa.variables.relative_x,
fpa.variables.relative_y])
fig = go.Figure(p_xy.plot(), p_xy.plotly_layout())

```

```
fig.update_layout(title="Marginal View of relative x and y position of the
robot with respect to the object.")
fig.show()
```

Next, we will perform pick up tasks using the default policy and observe the success rate. The robot will now experiment with the behaviour specified by the default policy and observe his success rate in doing so. After finishing the experiments, we insert the results into the database. If you want to generate some data locally, you can uncomment the following code.

```
# pycram.orm.base.ProcessMetaData().description = "Experimenting with Pick
Up Actions"
# fpa.sample_amount = 100
# with simulated_robot:
#     fpa.batch_rollout()
# task_tree.root.insert(session)
# session.commit()
# task_tree.reset_tree()
```

Let's query the data that is needed to learn a pick up action and have a look at it.

```
samples = pd.read_sql(fpa.query_for_database(), engine)
samples["arm"] = samples["arm"].astype(str)
samples["grasp"] = samples["grasp"].astype(str)
samples
```

We can now learn a probabilistic model from the data. We will use the JPT algorithm to learn a model from the data.

```
variables = infer_variables_from_dataframe(samples,
scale_continuous_types=False,
                                min_samples_per_quantile=2,
                                min_likelihood_improvement =
0.)
model = JPT(variables, min_samples_leaf=25)
model.fit(samples)
model = ProbabilisticCircuit.from_other(model)
print(model)

arm, grasp, relative_x, relative_y = model.variables
```

Let's have a look at how the model looks like. We will visualize the model density when we condition on grasping the object from the front with the left arm.

```
event = SimpleEvent({arm: Arms.LEFT, grasp:
Grasp.FRONT}).as_composite_set()
conditional_model, conditional_probability = model.conditional(event)
p_xy = conditional_model.marginal([relative_x, relative_y])
```

```
fig = go.Figure(p_xy.plot(), p_xy.plotly_layout())
fig.show()
```

Let's make a monte carlo estimate on the success probability of the new model.

```
fpa.policy = model
fpa.sample_amount = 5
with simulated_robot:
    fpa.batch_rollout()
```

We can see, that our new and improved model has a better success probability as opposed to the 30% from the standard policy.

Next, we put the learned model to the test in a complex environment, where the milk is placed in a difficult to access area.

```
from pycrap.ontologies import Apartment
kitchen = Object("apartment", Apartment, "apartment.urdf")

milk.set_pose(Pose([0.5, 3.15, 1.04]))
milk_description = ObjectDesignatorDescription(types=[Milk]).ground()
fpa = MoveAndPickUp(milk_description, arms=[Arms.LEFT, Arms.RIGHT],
                    grasps=[Grasp.FRONT, Grasp.LEFT, Grasp.RIGHT,
                             Grasp.TOP], policy=model)
fpa.sample_amount = 200

p_xy = model.marginal([relative_x, relative_y])
fig = go.Figure(p_xy.plot(), p_xy.plotly_layout())
fig.show()
```

Let's look at the density of the relative x and y position of the robot with respect to the milk. We can see that he would like to access the object from the right front area.

```
grounded_model = fpa.ground_model()
p_xy = grounded_model.marginal([relative_x, relative_y]).simplify()
fig = go.Figure(p_xy.plot(), p_xy.plotly_layout())
fig.update_layout(title="Marginal View of relative x and y position with
respect to the milk",
                  xaxis_range=[-1, 1], yaxis_range=[-1, 1])
fig.show()
```

Finally, we observe our improved plan in action.

```
from pycram.designators.action_designator import ParkArmsActionPerformable

world.reset_world()
milk.set_pose(Pose([0.5, 3.15, 1.04]))
torso_joint = RobotDescription.current_robot_description.torso_joint
```

```

with simulated_robot:
    MoveTorsoActionPerformable({torso_joint: 0.3}).perform()
    for sample in fpa:
        try:
            ParkArmsActionPerformable(Arms.RIGHT).perform()
            sample.perform()
            break
        except PlanFailure as e:
            continue

world.exit()
viz_marker_publisher._stop_publishing()

```

Action Designator

This example will show the different kinds of Action Designators that are available. We will see how to create Action Designators and what they do.

Action Designators are high-level descriptions of actions which the robot should execute.

Action Designators are created from an Action Designator Description, which describes the type of action as well as the parameter for this action. Parameter are given as a list of possible parameters. For example, if you want to describe the robot moving to a table you would need a [NavigateAction\(\)](#) and a list of poses that are near the table. The Action Designator Description will then pick one of the poses and return a performable Action Designator which contains the picked pose.

Navigate Action

We will start with a simple example of the [NavigateAction\(\)](#).

First, we need a BulletWorld with a robot.

```

from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import ObjectType, WorldMode
from pycram.datastructures.pose import Pose
from pycrap.ontologies import Robot, Milk, Apartment

world = BulletWorld(WorldMode.DIRECT)
pr2 = Object("pr2", Robot, "pr2.urdf", pose=Pose([1, 2, 0]))
apartment = Object("apartment", Apartment, "apartment.urdf")

```

```
milk = Object("milk", Milk, "milk.stl", pose=Pose([2.3, 2, 1.1]))
```

To move the robot we need to create a description and resolve it to an actual Designator. The description of navigation only needs a list of possible poses.

```
from pycram.designators.action_designator import NavigateAction
from pycram.datastructures.pose import Pose
```

```
pose = Pose([1.3, 2, 0], [0, 0, 0, 1])
```

```
# This is the Designator Description
navigate_description = NavigateAction(target_locations=[pose])
```

```
# This is the performable Designator
navigate_designator = navigate_description.resolve()
```

What we now did was: create the pose where we want to move the robot, create a description describing a navigation with a list of possible poses (in this case the list contains only one pose) and create an action designator from the description. The action designator contains the pose picked from the list of possible poses and can be performed.

```
from pycram.process_module import simulated_robot
```

```
with simulated_robot:
    navigate_designator.perform()
```

Every designator that is performed needs to be in an environment that specifies where to perform the designator either on the real robot or the simulated one. This environment is called [simulated_robot\(\)](#) similar there is also a [real_robot\(\)](#) environment.

There are also decorators which do the same thing but for whole methods, they are called [with real_robot\(\)](#) and [with simulated_robot\(\)](#).

Move Torso

This action designator moves the torso up or down, specifically it sets the torso joint to a given value.

We start again by creating a description and resolving it to a designator. Afterwards, the designator is performed in a [simulated_robot\(\)](#) environment.

```
from pycram.designators.action_designator import MoveTorsoAction
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import TorsoState
```

```
torso_pose = TorsoState.HIGH
```

```
torso_desig = MoveTorsoAction([torso_pose]).resolve()
```

```
with simulated_robot:
```

```
torso_desig.perform()
```

Set Gripper

As the name implies, this action designator is used to open or close the gripper.

The procedure is similar to the last time, but this time we will shorten it a bit.

```
from pycram.designators.action_designator import SetGripperAction
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import GripperState, Arms

gripper = Arms.RIGHT
motion = GripperState.OPEN

with simulated_robot:
    SetGripperAction(grippers=[gripper],
    motions=[motion]).resolve().perform()
```

Park Arms

Park arms is used to move one or both arms into the default parking position.

```
from pycram.designators.action_designator import ParkArmsAction
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import Arms

with simulated_robot:
    ParkArmsAction([Arms.BOTH]).resolve().perform()
```

Pick Up and Place

Since these two are dependent on each other, meaning you can only place something when you picked it up beforehand, they will be shown together.

These action designators use object designators, which will not be further explained in this tutorial so please check the example on object designators for more details.

To start we need an environment in which we can pick up and place things as well as an object to pick up.

```
world.reset_world()
from pycram.designators.action_designator import PickUpAction,
PlaceAction, ParkArmsAction, MoveTorsoAction, NavigateAction
from pycram.designators.object_designator import BelieveObject
from pycram.process_module import simulated_robot
```

```

from pycram.datastructures.enums import Arms, Grasp, TorsoState
from pycram.datastructures.pose import Pose

milk_desig = BelieveObject(names=["milk"])
arm = Arms.RIGHT

with simulated_robot:
    ParkArmsAction([Arms.BOTH]).resolve().perform()

    MoveTorsoAction([TorsoState.HIGH]).resolve().perform()

    NavigateAction([Pose([1.8, 2, 0.0],
                        [0.0, 0.0, 0., 1])]).resolve().perform()

    PickupAction(object_designator_description=milk_desig,
                  arms=[arm],
                  grasps=[Grasp.RIGHT]).resolve().perform()

    PlaceAction(object_designator_description=milk_desig,
                 target_locations=[Pose([2.4, 1.8, 1],
                                         [0, 0, 0, 1])],
                 arms=[arm]).resolve().perform()

```

Look At

Look at lets the robot look at a specific point, for example if it should look at an object for detecting.

```

world.reset_world()
from pycram.designators.action_designator import LookAtAction
from pycram.process_module import simulated_robot
from pycram.datastructures.pose import Pose

target_location = Pose([3, 2, 1], [0, 0, 0, 1])
with simulated_robot:
    LookAtAction(targets=[target_location]).resolve().perform()

```

Detect

Detect is used to detect objects in the field of vision (FOV) of the robot. We will use the milk used in the pick up/place example, if you didn't execute that example you can spawn the milk with the following cell. The detect designator will return a resolved instance of an `ObjectDesignatorDescription`.

```

world.reset_world()
from pycram.designators.action_designator import DetectAction,
LookAtAction, ParkArmsAction, NavigateAction

```

```

from pycram.designators.object_designator import BelieveObject
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import DetectionTechnique

milk_desig = BelieveObject(names=["milk"])

with simulated_robot:
    ParkArmsAction([Arms.BOTH]).resolve().perform()

    NavigateAction([Pose([1.7, 2, 0], [0, 0, 0, 1])]).resolve().perform()

    LookAtAction(targets=[milk_desig.resolve().pose]).resolve().perform()

    obj_desig =
DetectAction(DetectionTechnique.ALL, object_designator_description=milk_desig).resolve().perform()

    print(obj_desig)

```

Transporting

Transporting can transport an object from its current position to another target position. It is similar to the Pick and Place plan used in the Pick-up and Place example. Since we need an Object which we can transport we spawn a milk, you don't need to do this if you already have spawned it in a previous example.

```

world.reset_world()
from pycram.designators.action_designator import *
from pycram.designators.object_designator import *
from pycram.process_module import simulated_robot
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import Arms, TorsoState

milk_desig = BelieveObject(names=["milk"])

description = TransportAction(milk_desig,
                             [Pose([2.4, 1.8, 1],
                                    [0, 0, 0, 1])],
                             [Arms.LEFT])

with simulated_robot:
    MoveTorsoAction([TorsoState.HIGH]).resolve().perform()
    description.resolve().perform()

```

Opening

Opening allows the robot to open a drawer, the drawer is identified by an ObjectPart designator which describes the handle of the drawer that should be grasped.

For the moment this designator works only in the apartment environment, therefore we remove the kitchen and spawn the apartment.

```
world.reset_world()
from pycram.designators.action_designator import *
from pycram.designators.object_designator import *
from pycram.datastructures.enums import Arms, TorsoState
from pycram.process_module import simulated_robot
from pycram.datastructures.pose import Pose

apartment_desig = BelieveObject(names=["apartment"]).resolve()
handle_deisg = ObjectPart(names=["handle_cab10_t"],
part_of=apartment_desig)

with simulated_robot:
    MoveTorsoAction([TorsoState.HIGH]).resolve().perform()
    ParkArmsAction([Arms.BOTH]).resolve().perform()
    NavigateAction([Pose([1.7474915981292725, 2.6873629093170166, 0.0],
                        [-0.0, 0.0, 0.5253598267689507, -
0.850880163370435])]).resolve().perform()
    OpenAction(handle_deisg, [Arms.RIGHT]).resolve().perform()
```

Closing

Closing lets the robot close an open drawer, like opening the drawer is identified by an ObjectPart designator describing the handle to be grasped.

This action designator only works in the apartment environment for the moment, therefore we remove the kitchen and spawn the apartment. Additionally, we open the drawer such that we can close it with the action designator.

```
from pycram.designators.action_designator import *
from pycram.designators.object_designator import *
from pycram.datastructures.enums import Arms
from pycram.process_module import simulated_robot
from pycram.datastructures.pose import Pose

apartment_desig = BelieveObject(names=["apartment"]).resolve()
handle_deisg = ObjectPart(names=["handle_cab10_t"],
part_of=apartment_desig)

with simulated_robot:
    MoveTorsoAction([TorsoState.HIGH]).resolve().perform()
    ParkArmsAction([Arms.BOTH]).resolve().perform()
```

```

        NavigateAction([Pose([1.7474915981292725, 2.6873629093170166, 0.0],
                               [-0.0, 0.0, 0.5253598267689507, -
0.850880163370435]))).resolve().perform()
        CloseAction(handle_deisg, [Arms.RIGHT]).resolve().perform()
world.exit()

```

Object Designator

Object designators are used to describe objects located in the BulletWorld or the real environment and then resolve them during runtime to concrete objects.

Object designators are different from the Object class in `bullet_world.py` in the way that they just describe an object and do not create objects or provide methods to manipulate them. Nevertheless, object designators contain a reference to the BulletWorld object.

An Object designator takes two parameters, of which at least one has to be provided. These parameters are:

- A list of names
- A list of types

Object Designators work similar to Location designators, they get constraints describing a set of objects and when resolved return a specific instance.

For all following examples we need a BulletWorld, so let's create one.

```

from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import ObjectType, WorldMode
from pycram.datastructures.pose import Pose
world = BulletWorld(WorldMode.DIRECT)

```

Believe Object

This object designator is used to describe objects that are located in the BulletWorld. So objects that are in the belief state, hence the name. In the future when there is a perception interface, there will be a `RealObject` description which will be used to describe objects in the real world.

Since [`BelieveObject\(\)`](#) describes Objects in the BulletWorld we create a few.

```

from pycrap.ontologies import Milk, Cereal, Kitchen, Spoon
kitchen = Object("kitchen", Kitchen, "kitchen.urdf")

```

```

milk = Object("milk", Milk, "milk.stl", pose=Pose([1.3, 1, 0.9]))
cereal = Object("froot_loops", Cereal, "breakfast_cereal.stl",
pose=Pose([1.3, 0.9, 0.95]))
spoon = Object("spoon", Spoon, "spoon.stl", pose=Pose([1.3, 1.1, 0.87]))

```

Now that we have objects we can create an object designator to describe them. For the start we want an object designator only describing the milk. Since all objects have unique names we can create an object designator using a list with only the name of the object.

```

from pycram.designators.object_designator import BelieveObject

```

```

object_description = BelieveObject(names=["milk"])

```

```

print(object_description.resolve())

```

You can also use the type to describe objects, so now we want to have an object designator that describes every food in the world.

```

from pycram.designators.object_designator import BelieveObject

```

```

object_description = BelieveObject(types=[Milk, Cereal])

```

```

print(object_description.resolve())

```

Object Part

Part of object designators can be used to describe something as part of another object. For example, you could describe a specific drawer as part of the kitchen. This is necessary since the drawer is no single BulletWorld Object but rather a link of the kitchen which is a BulletWorld Object.

For this example we need just need the kitchen, if you didn't spawn it in the previous example you can spawn it with the following cell.

```

from pycram.designators.object_designator import ObjectPart, BelieveObject

```

```

kitchen_desig = BelieveObject(names=["kitchen"]).resolve()

```

```

object_description =
ObjectPart(names=["sink_area_left_upper_drawer_main"],
part_of=kitchen_desig)

```

```

print(object_description.resolve())

```

Object Designators as Generators

Similar to location designators object designators can be used as generators to iterate through every object that they are describing. We will see this at the example of an object designator describing every type of food.

For this we need some objects, so if you didn't already spawn them you can use the next cell for this.

```
from pycram.designators.object_designator import BelieveObject

object_description = BelieveObject(types=[Milk, Cereal])

for obj in object_description:
    print(obj, "\n")
```

To close the world use the following exit function.

```
world.exit()
```

Location Designator

This example will show you what location designators are, how to use them and what they are capable of.

Location Designators are used to semantically describe locations in the world. You could, for example, create a location designator that describes every position where a robot can be placed without colliding with the environment. Location designator can describe locations for:

- Visibility
- Reachability
- Occupancy
- URDF Links (for example a table)

To find locations that fit the given constraints, location designator create Costmaps.

Costmaps are a 2D distribution that have a value greater than 0 for every position that fits the costmap criteria.

Location designators work similar to other designators, meaning you have to create a location designator description which describes the location. This description can then be resolved to the actual 6D pose on runtime.

Occupancy

We will start with a simple location designator that describes a location where the robot can be placed without colliding with the environment. To do this we need a BulletWorld since the costmaps are mostly created from the current state of the BulletWorld.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import ObjectType, WorldMode
from pycram.datastructures.pose import Pose
from pycrap.ontologies import Apartment, Robot, Milk

use_multiverse = False
viz_marker_publisher = None
if use_multiverse:
    try:
        from pycram.worlds.multiverse import Multiverse
        world = Multiverse()
    except ImportError:
        raise ImportError("Multiverse is not installed, please install it
to use it.")
else:
    from pycram.ros_utils.viz_marker_publisher import VizMarkerPublisher
    world = BulletWorld()
    viz_marker_publisher = VizMarkerPublisher()

apartment = Object("apartment", Apartment, "apartment.urdf")
pr2 = Object("pr2", Robot, "pr2.urdf")
```

Next up we will create the location designator description, the [CostmapLocation\(\)](#) that we will be using needs a target as a parameter. This target describes what the location designator is for, this could either be a pose or object that the robot should be able to see or reach.

In this case we only want poses where the robot can be placed, this is the default behaviour of the location designator which we will be extending later.

```
from pycram.designators.location_designator import CostmapLocation

target = apartment.get_pose()

location_description = CostmapLocation(target)

pose = location_description.resolve()

print(pose)
```

Reachable

Next we want to have locations from where the robot can reach a specific point, like an object the robot should pick up. This can also be done with the [CostmapLocation\(\)](#) description, but

this time we need to provide an additional argument. The additional argument is the robot which should be able to reach the pose.

Since a robot is needed we will use the PR2 and use a milk as a target point for the robot to reach. The torso of the PR2 will be set to 0.2 since otherwise the arms of the robot will be too low to reach on the countertop.

```
pr2.set_joint_position("torso_lift_joint", 0.2)
milk = Object("milk", Milk, "milk.stl", pose=Pose([1.3, 1, 0.9]))

from pycram.designators.location_designator import CostmapLocation
from pycram.designators.object_designator import BelieveObject

target = BelieveObject(names=["milk"]).resolve()
robot_desig = BelieveObject(names=["pr2"]).resolve()

location_description = CostmapLocation(target=target,
reachable_for=robot_desig)

print(location_description.resolve())
```

As you can see we get a pose near the countertop where the robot can be placed without colliding with it. Furthermore, we get a list of arms with which the robot can reach the given object.

Visible

The [CostmapLocation\(\)](#) can also find position from which the robot can see a given object or location. This is very similar to how reachable locations are described, meaning we provide a object designator or a pose and a robot designator but this time we use the `visible_for` parameter.

For this example we need the milk as well as the PR2, so if you did not spawn them during the previous location designator you can spawn them with the following cell.

```
from pycram.designators.location_designator import CostmapLocation
from pycram.designators.object_designator import BelieveObject

target = BelieveObject(names=["milk"]).resolve()
robot_desig = BelieveObject(names=["pr2"]).resolve()

location_description = CostmapLocation(target=target,
visible_for=robot_desig)

print(location_description.resolve())
```

Semantic

Semantic location designators are used to create location descriptions for semantic entities, like a table. An example of this is: You have a robot that picked up an object and should place it on a table. Semantic location designator then allows to find poses that are on this table.

Semantic location designator need an object from which the target entity is a part and the URDF link representing the entity. In this case we want a position on the kitchen island, so we have to provide the kitchen object designator since the island is a part of the kitchen and the link name of the island surface.

For this example we need the kitchen as well as the milk. If you spawned them in one of the previous examples you don't need to execute the following cell.

```
from pycram.designators.location_designator import SemanticCostmapLocation
from pycram.designators.object_designator import BelieveObject

kitchen_desig = BelieveObject(names=["apartment"]).resolve()
milk_desig = BelieveObject(names=["milk"]).resolve()

counter_name = "counter_sink_stove" if use_multiverse else
"island_countertop"
location_description = SemanticCostmapLocation(link_name=counter_name,
                                              part_of=kitchen_desig,
                                              for_object=milk_desig)

print(location_description.resolve())
```

Location Designator as Generator

Location designator descriptions implement an iter method, so they can be used as generators which generate valid poses for the location described in the description. This can be useful if the first pose does not work for some reason.

We will see this at the example of a location designator for visibility. For this example we need the milk, if you already have a milk spawned in your world you can ignore the following cell.

```
from pycram.designators.location_designator import CostmapLocation
from pycram.designators.object_designator import BelieveObject

target = BelieveObject(names=["milk"]).resolve()
robot_desig = BelieveObject(names=["pr2"]).resolve()

location_description = CostmapLocation(target=target,
                                       visible_for=robot_desig)

for pose in location_description:
    print(pose.pose)
```

Accessing Locations

Accessing describes a location from which the robot can open a drawer. The drawer is specified by a `ObjectPart` designator which describes the handle of the drawer.

At the moment this location designator only works in the apartment environment, so please remove the kitchen if you spawned it in a previous example. Furthermore, we need a robot, so we also spawn the PR2 if it isn't spawned already.

```
from pycram.designators.object_designator import *
from pycram.designators.location_designator import *

apartment_desig = BelieveObject(names=["apartment"])
handle_name = "cabinet10_drawer1_handle" if use_multiverse else
"handle_cab10_t"
handle_desig = ObjectPart(names=[handle_name],
part_of=apartment_desig.resolve())
robot_desig = BelieveObject(types=[Robot])

access_location = AccessingLocation(handle_desig.resolve(),
robot_desig.resolve(),
                                prepose_distance=0.03).resolve()

print(access_location.pose)
```

Giskard Location

Some robots like the HSR or the Stretch2 need a full-body ik solver to utilize the whole body. For this case the [GiskardLocation\(\)](#) can be used. This location designator uses giskard as an ik solver to find a pose for the robot to reach a target pose.

Note: The `GiskardLocation` relies on Giskard, therefore Giskard needs to run in order for this Location Designator to work.

```
from pycram.ros import get_node_names
if "/giskard" in get_node_names():

    from
pycram.designators.specialized_designators.location.giskard_location
import GiskardLocation

    robot_desig = BelieveObject(names=["pr2"]).resolve()

    loc = GiskardLocation(target=Pose([1, 1, 1]),
reachability_for=robot_desig).resolve()
    print(loc.pose)
```

If you are finished with this example you can close the world with the following cell:

```

if viz_marker_publisher is not None:
    viz_marker_publisher._stop_publishing()
world.exit()

```

Motion Designator

Motion designators are similar to action designators, but unlike action designators, motion designators represent atomic low-level motions. Motion designators only take the parameter that they should execute and not a list of possible parameters, like the other designators. Like action designators, motion designators can be performed. Performing a motion designator verifies the parameter and passes the designator to the respective process module.

Since motion designators perform a motion on the robot, we need a robot which we can use. Therefore, we will create a BulletWorld as well as a PR2 robot.

```

from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import ObjectType, WorldMode
from pycram.datastructures.pose import Pose
from pycrap.ontologies import Robot, Milk

world = BulletWorld(WorldMode.DIRECT)
pr2 = Object("pr2", Robot, "pr2.urdf")
milk = Object("milk", Milk, "milk.stl", pose=Pose([1.5, 0, 1]))

```

Move

Move is used to let the robot drive to the given target pose. Motion designator are used in the same way as the other designator, first create a description then resolve it to the actual designator and lastly, perform the resolved designator.

```

from pycram.datastructures.pose import Pose
from pycram.designators.motion_designator import MoveMotion
from pycram.process_module import simulated_robot

with simulated_robot:
    motion_description = MoveMotion(target=Pose([1, 0, 0], [0, 0, 0, 1]))

    motion_description.perform()
world.reset_world()

```

MoveTCP

MoveTCP is used to move the tool center point (TCP) of the given arm to the target position specified by the parameter. Like any designator we start by creating a description and then resolving and performing it.

```
from pycram.designators.motion_designator import MoveTCPMotion
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import Arms

with simulated_robot:
    motion_description = MoveTCPMotion(target=Pose([0.5, 0.6, 0.6], [0, 0, 0, 1]), arm=Arms.LEFT)

    motion_description.perform()
```

Looking

Looking motion designator adjusts the robot state such that the cameras point towards the target pose. Although this motion designator takes the target as position and orientation, in reality only the position is used.

```
from pycram.designators.motion_designator import LookingMotion
from pycram.process_module import simulated_robot

with simulated_robot:
    motion_description = LookingMotion(target=Pose([1, 1, 1], [0, 0, 0, 1]))

    motion_description.perform()
```

Move Gripper

Move gripper moves the gripper of an arm to one of two states. The states can be [OPEN](#) and [CLOSE](#), which open and close the gripper respectively.

```
from pycram.designators.motion_designator import MoveGripperMotion
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import Arms, GripperState

with simulated_robot:
    motion_description = MoveGripperMotion(motion=GripperState.OPEN, gripper=Arms.LEFT)

    motion_description.perform()
```

Detecting

This is the motion designator implementation of detecting, if an object with the given object type is in the field of view (FOV) this motion designator will return a list of object designators describing the objects. It is important to specify the technique and state of the detection. You can also optional specify a region in which the object should be detected.

Since we need an object that we can detect, we will spawn a milk for this.

```
from pycram.designators.motion_designator import DetectingMotion,
LookingMotion
from pycram.process_module import simulated_robot
from pycram.datastructures.pose import Pose
from pycram.datastructures.enums import DetectionTechnique, DetectionState
from pycram.designators.object_designator import BelieveObject

with simulated_robot:
    LookingMotion(target=Pose([1.5, 0, 1], [0, 0, 0, 1])).perform()

    motion_description =
DetectingMotion(technique=DetectionTechnique.TYPES,state=DetectionState.ST
ART, object_designator_description=BelieveObject(types=[Milk]),
                region=None)

    obj = motion_description.perform()

    print(obj[0])
```

Move Arm Joints

This motion designator moves one or both arms. Movement targets are a dictionary with joint name as key and target pose as value.

```
from pycram.designators.motion_designator import MoveArmJointsMotion
from pycram.process_module import simulated_robot

with simulated_robot:
    motion_description =
MoveArmJointsMotion(right_arm_poses={"r_shoulder_pan_joint": -0.7})

    motion_description.perform()
```

World State Detecting

World state detecting is also used to detect objects, however, the object is not required to be in the FOV of the robot. As long as the object is somewhere in the belief state (BulletWorld) a resolved object designator will be returned.

Sine we want to detect something we will spawn an object that we can detect. If you already spawned the milk from the previous example, you can skip this step.

```
from pycram.designators.motion_designator import WorldStateDetectingMotion
from pycram.process_module import simulated_robot

with simulated_robot:
    motion_description = WorldStateDetectingMotion(object_type=Milk)

    obj = motion_description.perform()

    print(obj)
```

Move Joints

Move joints can move any number of joints of the robot, the designator takes two lists as parameter. The first list are the names of all joints that should be moved and the second list are the positions to which the joints should be moved.

```
from pycram.designators.motion_designator import MoveJointsMotion
from pycram.process_module import simulated_robot

with simulated_robot:
    motion_description = MoveJointsMotion(names=["torso_lift_joint",
"r_shoulder_pan_joint"], positions=[0.2, -1.2])

    motion_description.perform()
```

The following cell can be used after testing the examples, to close the BulletWorld.

```
world.exit()
```

ORM

When adding parameters to the Actions/Motions, you need to add those in the ORM classes as well. Make sure to set "init=False" and "default=default_value" in the mapped_column if the added parameter is not a required parameter.

Example:

```
class Action():

    new_parameter: float = 0.0

class ActionORM(Base):
```

```
new_parameter: Mapped[float] = mapped_column(init=False, default=0.0)
```

When modifying an action by adding another action/motion as part of it make sure to modify the position_results assertion in test_node in the test_orm.py and test_simulated_tree, test_tree_creation in test_task_tree.py accordingly. A helpful technique is to print the task_tree and count the poses, so each action has at least one pose since it has a robot_state unlike motions, then you add the poses that are needed in that action:

```
root = pycram.tasktree.task_tree.root
print(RenderTree(root, style=AsciiStyle()))
```

Hands on Object Relational Mapping in PyCram

This tutorial will walk you through the serialization of a minimal plan in pycram. First we will import sqlalchemy, create an in-memory database and connect a session to it.

```
import sqlalchemy.orm

engine = sqlalchemy.create_engine("sqlite+pysqlite:///memory:",
echo=False)
session = sqlalchemy.orm.Session(bind=engine)
session
```

Next we create the database schema using the sqlalchemy functionality. For that we need to import the base class of pycram.orm.

```
import pycram.orm.action_designator
pycram.orm.base.Base.metadata.create_all(engine)
session.commit()
```

Next we will write a simple plan where the robot parks his arms and then moves somewhere. We will construct a TaskTree around it such that we can serialize it later. As usual, we first create a world and then define the plan. By doing so, we obtain the task tree.

```
from pycram.designators.action_designator import *
from pycram.designators.location_designator import *
from pycram.process_module import simulated_robot
from pycram.datastructures.enums import Arms, ObjectType, Grasp,
WorldMode, TorsoState
```

```

from pycram.tasktree import with_tree
import pycram.tasktree
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.designators.object_designator import *
from pycram.datastructures.pose import Pose
from pycram.orm.base import ProcessMetaData
import anytree
from pycrap.ontologies import Robot, Kitchen, Milk, Cereal

world = BulletWorld(WorldMode.DIRECT)
pr2 = Object("pr2", Robot, "pr2.urdf")
kitchen = Object("kitchen", Kitchen, "kitchen.urdf")
milk = Object("milk", Milk, "milk.stl", pose=Pose([1.3, 1, 0.9]))
cereal = Object("cereal", Cereal, "breakfast_cereal.stl", pose=Pose([1.3,
0.7, 0.95]))
milk_desig = ObjectDesignatorDescription(names=["milk"])
cereal_desig = ObjectDesignatorDescription(names=["cereal"])
robot_desig = ObjectDesignatorDescription(names=["pr2"]).resolve()
kitchen_desig = ObjectDesignatorDescription(names=["kitchen"])

@with_tree
def plan():
    with simulated_robot:
        ParkArmsActionPerformable(Arms.BOTH).perform()
        MoveTorsoAction([TorsoState.MID]).resolve().perform()
        pickup_pose = CostmapLocation(target=cereal_desig.resolve(),
reachables_for=robot_desig).resolve()
        pickup_arm = pickup_pose.reachable_arms[0]

        NavigateAction(target_locations=[pickup_pose.pose]).resolve().perform()
        PickupAction(object_designator_description=cereal_desig,
arms=[pickup_arm], grasps=[Grasp.FRONT]).resolve().perform()
        ParkArmsAction([Arms.BOTH]).resolve().perform()

        place_island = SemanticCostmapLocation("kitchen_island_surface",
kitchen_desig.resolve(),

cereal_desig.resolve()).resolve()

        place_stand = CostmapLocation(place_island.pose,
reachables_for=robot_desig, reachable_arm=pickup_arm).resolve()

        NavigateAction(target_locations=[place_stand.pose]).resolve().perform()

        PlaceAction(cereal_desig, target_locations=[place_island.pose],
arms=[pickup_arm]).resolve().perform()

```

```

        ParkArmsActionPerformable(Arms.BOTH).perform()

plan()

# set description of what we are doing
ProcessMetaData().description = "Tutorial for getting familiar with the
ORM."
task_tree = pycram.tasktree.task_tree
print(anytree.RenderTree(task_tree.root))

```

Next we serialize the task tree by recursively inserting from its root.

```
task_tree.root.insert(session)
```

We can look at our experiment (Process)MetaData to get some context on the data we just created.

```

from sqlalchemy import select

print(*session.scalars(select(pycram.orm.base.ProcessMetaData)).all())

```

Lastly we can look at various table to see how the structures got logged. For example, we can get all the navigate actions that occurred.

```

navigations =
session.scalars(select(pycram.orm.action_designator.NavigateAction)).all()
print(*navigations, sep="\n")

```

Due to the inheritance mapped in the ORM package, we can also obtain all executed actions with just one query.

```

actions =
session.scalars(select(pycram.orm.action_designator.Action)).all()
print(*actions, sep="\n")

```

Of course all relational algebra operators, such as filtering and joining also work in pycram.orm queries. Let's say we need all the poses of objects, that were picked up by a robot. Since we defined a relationship between the PickupAction table and the Object table and between the Object table and the Pose table in the ORM class schema, we can just use the join operator without any further specification:

```

object_actions = (session.scalars(select(pycram.orm.base.Pose)
                                .join(pycram.orm.action_designator.PickUpAction.object)
                                .join(pycram.orm.object_designator.Object.pose))
                  .all())
print(*object_actions, sep="\n")

```

Did you notice, that for the joins we did not join the tables together in a typical sql kind of way, but rather used the relationships defined in the ORM classes and wrote joins like `PickUpAction.object` or `Object.pose`? This is because the ORM package automatically creates the joins for us, so we only have to join on the attributes that hold the relationship. This is a huge advantage over writing sql queries by hand, since we do not have to worry about the join conditions. This is a strong tool, but it is crucial to use it properly. Very important to note: The order of the joins matters! For instance, if we joined the Pose table with the Object table first, and placed the join between the PickUpAction table and the Object table second, sqlalchemy would have selected the Pose not from the join between all three tables, but rather from a join between the Pose and the Object table + from a join between the PickUpAction table and the Object table. These mistakes can lead to wrong results or even to errors (the above-mentioned example would actually lead to an error due to the Object table being accessed twice in two separate joins in the same query and therefore the column names of the Object tables would have been ambiguous and could not be used by sqlalchemy to join).

Make sure to check out the other examples of ORM querying.

If we want to filter for all successful tasks we can just add the filter operator:

```
from pycram.orm.tasktree import TaskTreeNode

successful_tasks =
session.scalars(select(TaskTreeNode).where(TaskTreeNode.status ==
"SUCCEEDED"))
print(*successful_tasks, sep="\n")
```

As expected all but the root node succeeded, since the root node is still running.

Writing an extension to the ORM package is also done with ease. We need to create a new ActionDesignator class and its ORM equivalent, where we define our new table. Let's say we want to log all the things the robot says. We will create a new ActionDesignator class called Saying and its ORM equivalent called ORMSaying.

```
from sqlalchemy.orm import Mapped, mapped_column, Session
from pycram.orm.action_designator import Action
from dataclasses import dataclass

# define ORM class from pattern in every pycram.orm class
class ORMSaying(Action):
```

```

    id: Mapped[int] =
mapped_column(sqlalchemy.ForeignKey(f'{Action.__tablename__}.id'),
primary_key=True, init=False)
    # since we do not want to add any custom specifications to our column,
we don't even need to define mapped_column, sqlalchemy does this
internally.
    text: Mapped[str]

# define brand new action designator
# Since this class is derived from ActionAbstract, we do not need to
manually define the insert() and to_sql() function, the mapping is done
automatically. We just have to tell the class, which ORMclass it is
supposed to use.
@dataclass
class SayingActionPerformable(ActionAbstract):

    text: str
    orm_class = ORMSaying

    @with_tree
    def plan(self) -> None:
        print(self.text)

    def validate(self, result=None):
        pass

```

Now we got our new ActionDesignator called Saying and its ORM version. Since this class got created after all other classes got inserted into the database (in the beginning of the notebook) we have to insert it manually.

```
ORMSaying.metadata.create_all(bind=engine)
```

Now we can create and insert a Saying action. Since this is the last part where we interact with the BulletWorld, we can also close it.

```

# create a saying action and insert it
SayingActionPerformable("Patchie, Patchie; Where is my
Patchie?").perform()
pycram.tasktree.task_tree.root.insert(session)
session.commit()

world.exit()

```

It is notable that committing the object to the session fills its primary key. Hence, there is no worries about assigning unique IDs manually. Finally, we can double-check that our object exists in the database.

```
session.scalars(select(ORMSaying)).all()
```

ORM querying

Querying the ORM can be done by writing custom queries using sqlalchemy. However, if you regularly want to access a chunk of data, it is recommended to write a view that does this for you. You can see the details of writing a view in [here](#)

```
:py:func:pycram.orm.base.views.PickUpWithContextView.
```

Migrate NEEMs

In this tutorial we will go through the process of migrating locally stored PyCRORM NEEMs to an already existing PyCRORM NEEM-Hub.

In some cases it may occur that you want to record data from a pycram controlled robot locally and perform some local actions before migrating your data to a big database server. In such cases, you can easily make a local database and connect your pycram process to it.

After you recorded your data locally you can migrate the data using the `migrate_neems` function.

First, let's create an in-memory database engine called `source_engine` where we record our current process.

```
import sqlalchemy.orm
import pycram

source_engine: sqlalchemy.engine.Engine
source_engine = sqlalchemy.create_engine("sqlite+pysqlite:///memory:",
echo=False)
source_session_maker = sqlalchemy.orm.sessionmaker(bind=source_engine)
pycram.orm.base.Base.metadata.create_all(source_engine) #create all Tables
Next, create an engine called destination_engine for the destination database where you want to migrate your NEEMs to. Note: This is just an example configuration.
```

```
destination_engine: sqlalchemy.engine.Engine
destination_engine =
sqlalchemy.create_engine("postgresql+psycopg2://alice:alice123@localhost:5
433/pycram", echo=False) # example values
```

```
destination_session_maker =  
sqlalchemy.orm.sessionmaker(bind=destination_engine)
```

If you already have some data in your local database you can skip the next block,
otherwise we will quickly create some example data

```
from pycram.datastructures.enums import Arms, ObjectType  
from pycram.designators.action_designator import *  
from pycram.designators.location_designator import *  
from pycram.process_module import simulated_robot  
from pycram.tasktree import with_tree  
from pycram.worlds.bullet_world import BulletWorld  
from pycram.world_concepts.world_object import Object  
from pycram.designators.object_designator import *  
from pycrap.ontologies import Robot, Kitchen, Milk, Cereal  
  
class ExamplePlans:  
    def __init__(self):  
        self.world = BulletWorld("DIRECT")  
        self.pr2 = Object("pr2", Robot, "pr2.urdf")  
        self.kitchen = Object("kitchen", Kitchen, "kitchen.urdf")  
        self.milk = Object("milk", Milk, "milk.stl", pose=Pose([1.3, 1,  
0.9]))  
        self.cereal = Object("cereal", Cereal, "breakfast_cereal.stl",  
pose=Pose([1.3, 0.7, 0.95]))  
        self.milk_desig = ObjectDesignatorDescription(names=["milk"])  
        self.cereal_desig = ObjectDesignatorDescription(names=["cereal"])  
        self.robot_desig =  
ObjectDesignatorDescription(names=["pr2"]).resolve()  
        self.kitchen_desig =  
ObjectDesignatorDescription(names=["kitchen"])  
  
    @with_tree  
    def pick_and_place_plan(self):  
        with simulated_robot:  
            ParkArmsAction([Arms.BOTH]).resolve().perform()  
            MoveTorsoAction([0.3]).resolve().perform()  
            pickup_pose =  
CostmapLocation(target=self.cereal_desig.resolve(),  
reachable_for=self.robot_desig.resolve())  
            pickup_arm = pickup_pose.reachable_arms[0]  
  
            NavigateAction(target_locations=[pickup_pose.pose]).resolve().perform()  
            PickupAction(object_designator_description=self.cereal_desig,  
arms=[pickup_arm],  
grasps=["front"]).resolve().perform()  
            ParkArmsAction([Arms.BOTH]).resolve().perform()
```

```

        place_island =
SemanticCostmapLocation("kitchen_island_surface",
self.kitchen_desig.resolve(),

self.cereal_desig.resolve()).resolve()

        place_stand = CostmapLocation(place_island.pose,
reachable_for=self.robot_desig,

reachable_arm=pickup_arm).resolve()

NavigateAction(target_locations=[place_stand.pose]).resolve().perform()

        PlaceAction(self.cereal_desig,
target_locations=[place_island.pose],
arms=[pickup_arm]).resolve().perform()

        ParkArmsAction([Arms.BOTH]).resolve().perform()

import pycram.orm.utils
import pycram.tasktree

with source_session_maker() as session:
    example_plans = ExamplePlans()
    for i in range(3):
        try:
            print("ExamplePlans run {}".format(i))
            example_plans.pick_and_place_plan()
            example_plans.world.reset_bullet_world()
            process_meta_data = pycram.orm.base.ProcessMetaData()
            process_meta_data.description = "Example Plan {}".format(i)
            process_meta_data.insert(session)
            pycram.tasktree.task_tree.root.insert(session)
            process_meta_data.reset()
        except Exception as e:
            print("Error: {}\n{}".format(type(e).__name__, e))
    session.commit()
    example_plans.world.exit()

```

Now that we have some example data or already had some example data all we need to do is migrate it over to the already existing PyCRORM NEEM-Hub.

```

pycram.orm.utils.migrate_neems(source_session_maker,destination_session_ma
ker)

```

If the command ran successfully the content of the source database should now be copied within the destination database. For example if we query for all the different meta_data, the previously defined instance comes up.

```

with destination_session_maker() as session:
    statement =
sqlalchemy.select('*').select_from(pycram.orm.base.ProcessMetaData)
    result = session.execute(statement).all()
    for item in result:
        print(item)

```

Pose

Poses in PyCRAM are represented by the Pose class which inherits from the PoseStamped message of ROS. This makes PyCRAMs poses compatible with everything in ROS like services, topics or TF.

This notebook will provide an overview about poses, how to use them and what they can do. We will start by simply creating a pose.

Before we start a few words about naming convention of Poses in PyCRAM. Naming convention is similar to the PoseStamped message so if you are familiar with that this should be easy.

- **Position:** A position means the position in cartesian space, so the x, y, and z coordinates.
- **Orientation:** An orientation is the rotation in all three axes represented as a quaternion with x, y, z, w.
- **Pose:** A pose is the combination of a position and an orientation. Poses in PyCRAM also contain a frame of reference to which the position and orientation are relative.

```

from pycram.datastructures.pose import Pose

```

```

example_pose = Pose([1, 2, 3], [0, 0, 0, 1], "map")
print(example_pose)

```

As you can see we created the `example_pose` with a position of `[1, 2, 3]` and an orientation of `[0, 0, 0, 1]` in the frame `map`. But we don't need to provide all these parameters for a Pose, in case there is no parameter the Pose will use default parameter.

```

from pycram.datastructures.pose import Pose

```

```

default_pose = Pose()
print(default_pose)

```

In case no parameter is provided the default parameter are:

- position: `[0, 0, 0]`
- orientation: `[0, 0, 0, 1]`
- frame: `map`

The following example will show how to access the data stored in a pose.

```

from pycram.datastructures.pose import Pose

```

```

example_pose = Pose([1, 2, 3], [0, 0, 0, 1], "map")

print(f"Access to a component of the position: {example_pose.position.y}")

print(f"Access to a component of the rotation:
{example_pose.orientation.x}")

print(f"Get the whole position as
geometry_msgs/Pose:\n{example_pose.position}")

print(f"You can also get position or orientation as a list:
{example_pose.position_as_list()}")

print(f"Same with the whole pose: {example_pose.to_list()}")

print(f"Access the reference frame: {example_pose.frame}")

```

Editing a pose

You can also edit the data saved in a Pose, similar to how you access it.

```

from pycram.datastructures.pose import Pose

example_pose = Pose([1, 2, 3], [0, 0, 0, 1], "map")

# Edit a single component of the position
example_pose.position.x = 3
print(f"Edit only one component:\n{example_pose.position}", "\n")

# Edit the whole position
example_pose.position = [0, 0, 1]
print(f"Edit the whole position:\n{example_pose.position}", "\n")

example_pose.frame = "new_frame"
print(f"Set a new frame:\n{example_pose.frame}", "\n")

example_pose.set_position([3, 2, 1])
print(f"Set the position via method:\n{example_pose.position}", "\n")

```

Copy Poses

You can also copy Poses to create a new Pose with the same data. This can be useful if you have a method which would need to alter the Pose, since poses are passed by reference to a method every change done to the Pose in the method would affect the instanced passed to the method.

```

from pycram.datastructures.pose import Pose

```

```
example_pose = Pose([1, 2, 3], [0, 0, 0, 1], "map")

copy_pose = example_pose.copy()

print(example_pose, "\n")
print(copy_pose)
```

Convert to Transform

PyCRAM also has its own transform at which we will take a look in the next section. However, here we will take a look at how to convert a Pose into a Transform.

For this example we will take a Pose which represents the current pose of a milk object and convert it into a Transform which represents the transformation from the `map` frame to the `milk` frame.

```
from pycram.datastructures.pose import Pose

milk_pose = Pose([3, 4, 1], [1, 0, 0, 1], "map")

milk_transform = milk_pose.to_transform("milk")

print(milk_transform)
```

Transforms

Transforms are similar to Poses but instead of representing a Pose in a frame of reference they represent a transformation from one frame of reference to another. For this purpose Transforms have an additional parameter called `child_frame_id` which is the frame of reference to which the Transform is pointing.

Transforms in PyCRAM inherit from the TransformStamped message of ROS which makes them, like Poses, compatible to ROS services and topics that expect a TransformStamped message. Therefore, the naming conventions of Transforms are the same as of TransformStamped which.

- **Translation:** The vector describing the transformation in cartesian space.
- **Rotation:** The quaternion describing the transformation of rotation.
- **Transform:** The combination of translation and rotation

```
from pycram.datastructures.pose import Transform

example_transform = Transform([1, 2, 2], [0, 0, 0, 1], "map", "object")

print(example_transform)
```

Transforms have the same methods to get and set values as Poses have, therefore only a short showcase will be given. For more details please look at the Pose example or the API documentation.

```
from pycram.datastructures.pose import Transform

example_transform = Transform([2, 5, 1], [0, 0, 1, 1], "map", "object")

print(f"Access the rotation:\n{example_transform.rotation}", "\n")

print(f"Access the child_frame: {example_transform.child_frame_id}", "\n")

# changing translation and rotation is exactly like with Poses.

example_transform.translation = [1, 1, 1]
print(f"New translation:\n{example_transform.translation}")
```

Convert to Pose and Copy

Analog to Poses Transforms have a method that converts a Transform to a Pose, in this process the `child_frame_id` will be lost.

Also like in Poses Transforms have a `copy` method which creates an exact copy of this Transform.

```
from pycram.datastructures.pose import Transform

milk_transform = Transform([1, 1, 1], [0, 0, 0, 1], "map", "milk")

milk_pose = milk_transform.to_pose()

print(f"The converted pose:\n{milk_pose}", "\n")

example_transform = Transform([1, 1, 1], [0, 0, 0, 1], "map", "milk")

copy_transform = example_transform.copy()

print(f"The copied transform:\n{copy_transform}")
```

Operations on Transforms

Transforms have, unlike Poses, operations that can be done. These operations are:

- Multiplication
- Invert
- InverseTimes

Multiplication

We will first take a look at the multiplication of Transforms. We will use an example where we have two Transforms, the first from map to a hand frame and the second from the hand to a milk frame. By multiplying these two we get the Transform from map to milk frame.

```
from pycram.datastructures.pose import Transform

map_to_hand = Transform([1, 1, 1], [0, 0, 0, 1], "map", "hand")

hand_to_milk = Transform([0.1, 0.05, 0], [0, 0, 0, 1], "hand", "milk")

map_to_milk = map_to_hand * hand_to_milk

print(map_to_milk)
```

Invert

This inverts a Transform, so if we have a transform from map to milk then inverting it results in a Transform from milk to map.

```
from pycram.datastructures.pose import Transform

map_to_milk = Transform([1, 1, 0.5], [0, 0, 0, 1], "map", "milk")

milk_to_map = map_to_milk.invert()

print(milk_to_map)
```

Inverse Times

Inverse times combines the inverting and multiplication of Transforms, this results in a ‘minus’ for Transforms. We will again use the example of a hand holding a milk, but this time we have the Transforms from map to milk and hand to milk.

```
from pycram.datastructures.pose import Transform

map_to_milk = Transform([1.1, 1.05, 1], [0, 0, 0, 1], "map", "milk")

hand_to_milk = Transform([0.1, 0.05, 0], [0, 0, 0, 1], "hand", "milk")

map_to_milk = map_to_milk.inverse_times(hand_to_milk)

print(map_to_milk)
```

Robot Description

The robot description contains semantic information about the robot which can not be extracted from the URDF in a general way. This includes kinematic chains, end-effectors, cameras and their parameter, etc.

In general a Robot Description consists a number of different descriptions, these are:

- RobotDescription
- KinematicChainDescription
- EndEffectorDescription
- CameraDescription

In this example we will create a robot description step-by-step and describe the different components on the way. The robot we will use as an example will be the PR2, the complete PR2 description can also be seen

in [pycram.robot_descriptions.pr2_description\(\)](#).

Robot Description Class

We start by creating an instance of the [RobotDescription](#) class, this will serve as a the main component to which all other descriptions will be added.

To initialize a [RobotDescription](#) we need a few parameter which are:

- Name
- base_link
- torso_link
- torso_joint
- Path to a URDF file

```
from pycram.robot_description import RobotDescription
from pycram.ros import get_ros_package_path

filename = get_ros_package_path('pycram') + '/resources/robots/' + "pr2" +
'.urdf'
pr2_description = RobotDescription("pr2_example", "base_link",
"torso_lift_link", "torso_lift_joint", filename)
```

Kinematic Chain Description

The kinematic chain description describes a chain of links and joints of the robot which might be interesting when working with the robot. An example of such a chain would be the arm of the

robot, when programming for the robot it is important to know which links and joints exactly make up the arm, however, these can not be extracted from the URDF automatically.

The kinematic chain is based upon the URDF, meaning when initializing the description one only needs to specify the first and last link of the chain.

We will now create the kinematic chain description for the right arm of the PR2. For initializing the [KinematicChainDescription](#) the following parameter are needed:

- Name
- first link
- last link
- URDF object
- Arm type

The arm type specifies which arm this kinematic chain describes, this is needed when one wants to access only the arms of the robot.

```
from pycram.robot_description import KinematicChainDescription
from pycram.datastructures.enums import Arms

right_arm = KinematicChainDescription("right", "torso_lift_link",
                                     "r_wrist_roll_link",
                                     pr2_description.urdf_object,
                                     arm_type=Arms.RIGHT)
```

The created [KinematicChainDescription](#) can now be added to the robot description.

```
pr2_description.add_kinematic_chain_description(right_arm)
```

End Effector Description

Since kinematic chains only describe a moveable chain of links and joints like arms these do not represent end-effectors which can be used to do manipulation tasks.

To represent end-effectors we will create an [EndEffectorDescription](#) which contains the information of the respective end-effector. When creating an [EndEffectorDescription](#) we need the following parameter:

- Name
- first link
- tool_frame
- URDF object

You might have noticed that the end-effector only has a first link but no last link, this is the case since end-effectors are at the end of the arms. Therefore, all links and joints below a certain link can be seen as part of the end-effector.

```
from pycram.robot_description import EndEffectorDescription
```

```
right_gripper = EndEffectorDescription("right_gripper",
    "r_gripper_palm_link", "r_gripper_tool_frame",
    pr2_description.urdf_object)
```

The gripper can now be added to the previously created [KinematicChainDescription](#).

```
right_arm.end_effector = right_gripper
```

Camera Description

The camera description contains all parameters of a camera, which is mounted on the robot. The parameters for the [CameraDescription](#) are:

- Name
- Link name
- minimal height
- maximal height
- horizontal angle
- vertical angle

```
from pycram.robot_description import CameraDescription
from pycram.datastructures.enums import Grasp
```

```
camera = CameraDescription("kinect_camera", "wide_stereo_optical_frame",
    1.27,
    1.60, 0.99483, 0.75049)
```

The finished camera description can now be added to the robot description.

```
pr2_description.add_camera_description(camera)
```

Grasps

Grasps define how a robot interacts with objects. The grasps defined in the robot description define for each grasp (right, left, top, front) the orientation of the end-effector, relative to the base_frame of the robot, to achieve the respective grasp.

```
pr2_description.add_grasp_orientations({Grasp.FRONT: [0, 0, 0, 1],
    Grasp.LEFT: [0, 0, -1, 1],
    Grasp.RIGHT: [0, 0, 1, 1],
    Grasp.TOP: [0, 1, 0, 1]})
```

Register Robot Description

Lastly, you need to register the robot description to the [RobotDescriptionManager](#). As you can see the code to register the robot description has to be executed at the start of PyCRAM, if

you put your file with the robot description in the [pycram.robot_descriptions](#) directory it will be executed upon the start of PyCRAM.

```
from pycram.robot_description import RobotDescriptionManager

rdm = RobotDescriptionManager()
rdm.register_description(pr2_description)
```

Giskard interface in PyCRAM

This notebook should provide you with an example on how to use the Giskard interface. This includes how to use the interface, how it actually works, and how to extend it.

We start by installing and launching Giskard. For the installation please follow the instructions [here](#). After you finish the installation you should be able to launch Giskard by calling:

```
roslaunch giskardpy giskardpy_pr2_standalone.launch
```

This way you can launch Giskard for any robot that is supported:

```
roslaunch giskardpy giskardpy_hsr_standalone.launch
```

"Standalone" means that Giskard only uses a simulated robot and does not look for a real robot. If you want to use Giskard with a real robot you have to switch out "standalone" with "iai", e.g:

```
roslaunch giskardpy giskardpy_hsr_iai.launch
```

To see what Giskard is doing you can start RViz, there should already be a MarkerArray when starting otherwise you have to add this manually.

How to use the Giskard interface

Everything related to the Giskard interface is located in [pycram.external_interfaces.giskard](#). The content of the file can be roughly divided into three parts:

1. Methods to manage the belief states between PyCRAM and Giskard
2. Motion goals that should be sent to Giskard for execution
3. Helper methods to construct ROS messages

The most useful methods are the ones for sending and executing Motion goals. These are the ones we will mostly look at.

We will now start by setting up PyCRAM and then try to send some simple motion goals.

```
from pycram.bullet_world import BulletWorld, Object
from pycram.enums import ObjectType
```

```
world = BulletWorld()
```

```
pr2 = Object("pr2", ObjectType.ROBOT, "pr2.urdf")
```

When you are working on the real robot you also need to initialize the RobotStateUpdater, this module updates the robot in the BulletWorld with the pose and joint state of the real robot.

You might need to change to topic names to fit the topic names as published by your robot.

```
from pycram.ros_utils.robot_state_updater import RobotStateUpdater
```

```
r = RobotStateUpdater("/tf", "/joint_states")
```

Now we have a PyCRAM belief state set up, belief state in this case just refers to the BulletWorld since the BulletWorld represents what we believe the world to look like.

The next step will be to send a simple motion goal. The motion goal we will be sending is moving the torso up. For this we just need to move one joint, so we can use the `achieve_joint_goal`. This method takes a dictionary with the joints that should be moved and the target value for the joints.

Look at RViz to see the robot move, since we call Giskard for movement the robot in the BulletWorld will not move.

```
from pycram.external_interfaces import giskard
```

```
giskard.achieve_joint_goal({"torso_lift_joint": 0.28})
```

For Giskard everything is connected by joints (this is called a [World Tree](#) by Giskard) therefore we can move the robot's base by using motion goals between the map origin and the robot base. (e.g. by sending a "base_link" goal in the "map" frame).

In the example below we use a cartesian goal, meaning we give Giskard a goal pose, a root link and a tip link and Giskard tries to move all joints between root link and tip link such that the tip link is at the goal pose.

This sort of movement is fine for short distances, but keep in mind that Giskard has no collision avoidance for longer journeys. So using MoveBase for longer distances is a better idea.

```
from pycram.external_interfaces import giskard
from pycram.pose import Pose
```

```
giskard.achieve_cartesian_goal(Pose([1, 0, 0]), "base_link", "map")
```

Now for the last example: we will move the gripper using full body motion control.

We will again use the cartesian goal, but now between "map" and "r_gripper_tool_frame" frames. This will not only move the robot (because the kinematic chain between "map" and

“base_link” as used in the previous example is also part of this chain) but also move the arm of the robot such that it reaches the goal pose.

```
from pycram.external_interfaces import giskard
from pycram.pose import Pose

giskard.achieve_cartesian_goal(Pose([1, 0.5, 0.7]),
                                "r_gripper_tool_frame", "map")
```

That concludes this example you can now close the BulletWorld by using the “exit” method.

```
world.exit()
```

How the Giskard interface works

The PyCRAM interface to Giskard mostly relies on the Python interface that Giskard already provides ([tutorial](#) and the [source code](#)). This interface provides methods to achieve motion goals and load things into the Giskard believe state.

What PyCRAM does with this, is: Synchronize the belief state of Giskard with the one of PyCRAM by loading the environment URDF in Giskard, this is done before any motion goal is sent. Furthermore, the motion goals are wrapped in methods that use PyCRAM data types.

You can also set collisions between different groups of links. By default, Giskard avoids all collisions but for things like grasping an object you want to allow collisions of the gripper. The interface also supports the following collision modes:

- `avoid_all_collisions`
- `allow_self_collision`
- `allow_gripper_collision` The collision mode can be set by calling the respective method, after calling the method the collision mode is valid for the next motion goal. Afterwards, it defaults back to `avoid_all_collisions`.

There is a `init_giskard_interface` method which can be used as a decorator. This decorator should be used on all methods that access the `giskard_wrapper`, since it assures that the interface is working and checks if Giskard died or the imports for the `giskard_msgs` failed.

Extend the Giskard interface

At the moment the PyCRAM Giskard interface is mostly a wrapper around the Python interface provided by Giskard. If you want to extend the interface there are two ways:

- Wrap more motion goals which are provided by the Python interface
- Design new Higher-Level motion goals by combining the motion goals already provided

Robokudo interface in PyCRAM

This notebook should give you an example on how the RoboKudo interface in PyCRAM works. We will go over how to use the interface, how it is implemented and what can be extended.

First, you need to install RoboKudo by following the installation instructions [here](#).

RoboKudo depends on a pipeline of so-called annotators to process images, depending on your use-case the used annotators will change. But for this simple example we can use the demo pipeline from the [tutorial](#). You can start RoboKudo by calling

```
roslaunch robokudo main.py _ae=query
```

To get a stream of images to process you need the test bag file, from [here](#). You can run this bag file with the following command in the directory where the bag file is.

```
roslaunch test test.bag --loop
```

There should now be two windows which show you the result of the annotators. You switch between different annotators by using the arrow keys.

How to use the RoboKudo interface in PyCRAM

Everything related to the RoboKudo interface can be found in the file [pycram.external_interfaces.robokudo](#). The most important method of this file is `query()` which takes a PyCRAM object designator and calls RoboKudo to try to find a fitting object in the camera view. The other methods are just helper for constructing messages.

Since we are only working with the demo pipeline we will only see how the interface functions but not actually perceive objects in the images.

```
from pycram.external_interfaces import robokudo
from pycram.designators.object_designator import *
from pycram.enums import ObjectType
```

```
object_desig_desc = ObjectDesignatorDescription(types=[ObjectType.BOWL])
robokudo.query(object_desig_desc)
```

There was no object detected since the pipeline we are using for this example only returns an empty message. However, this should give you an impression on how the interface works.

How the RoboKudo interface in PyCRAM works

The interface to RoboKudo is designed around the ROS service that RoboKudo provides. The interface takes an ObjectDesignatorDescription which is PyCRAMs symbolic representation of objects and converts it to a RoboKudo ObjectDesignator, the RoboKudo ObjectDesignator is then send to RoboKudo.

The result from this is a list of RoboKudo ObjectDesignators which are possible matches that were found in the camera FOV. Each of these ObjectDesignators has a list of possible poses that are the result of different pose estimators (currently PyCRAM picks the pose from ‘ClusterPoseBBAnnotator’ from the list of possible poses). PyCRAM then transforms all possible poses for the found Objects to ‘map’ frame and returns them as a dictionary.

When using the interface the decorator [`init robokudo interface\(\)`](#) should be added to all methods that want to send queries to RoboKudo. This decorator makes sure that RoboKudo is running and creates an action client which can be used via the global variable `robokudo_action_client`.

How to extend the RoboKudo interface in PyCRAM

At the moment the RoboKudo interface is tailored towards a specific scenario, in which only two types of objects need to be detected. The distinction is mainly made by the difference in color, which is written in the RoboKudo ObjectDesignator depending on the ObjectType of the PyCRAM ObjectDesignator.

The main point for extension would be to make the interface more universal and extend it to work with other pipelines for example for human detection.

Ontologies in PyCRAM

Cognitive Architectures often include some logical theory to them that enables higher level reasoning, and so does PyCRAM. The PyCRAM typology PyCRAP is an ontology that can be used in PyCRAM to describe the belief of the robot using concepts. These individuals for the ontology are created on the fly, and PyCRAP can then be used to for instance filter for certain objects.

PyCRAP is defined in its package next to the PyCRAM package and hence allows users and developers to add new classes, instances, and relationships on demand. Furthermore, this architecture allows the users to directly see what’s in the ontology and how it is structured without switching to tools like [Protégé](#). The linter and AI assistants like Copilot can also deal with this notation and guide the users without annoyances.

PyCRAP contains a subpackage with ontologies. Every package in the ontologies subpackage is a separate ontology. The classes, properties, and so on can just be included via the import statement.

PyCRAP also comes with a parser to reflect any ontology that can be loaded with owlready2 in python files. This is very similar to the [code generation tool of sqlalchemy](#).

Note that this area is under construction and may frequently change.

Usage

You can access the ontology by importing the PyCRAP package:

```
import pycram
```

The ontology is structured in classes and instances. The classes are the concepts that are used to describe the world. The individuals in the ontology can be created when spawning objects in the world. For example, when you spawn a milk or a cereal box.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.datastructures.enums import WorldMode
from pycram.world_concepts.world_object import Object
from pycram.datastructures.pose import Pose
from pycrap.ontologies import Milk, Cereal, Food
```

```
world = BulletWorld(mode=WorldMode.DIRECT)
```

```
milk = Object("milk", Milk, "milk.stl")
cereal = Object("cereal", Cereal, "breakfast_cereal.stl", pose=Pose([1.4,
1, 0.95]))
```

You can query the ontology using [owlready2](#). For example, we can see all food objects like this:

```
print("All food instances", list(world.ontology.search(type=Food)))
```

You can also search for objects in the world using the ontology:

```
from pycram.designators.object_designator import
OntologyObjectDesignatorDescription
object_designator =
OntologyObjectDesignatorDescription(world.ontology.search(type=Food))
result_in_world = list(object_designator.__iter__())
print(result_in_world)
```

Architecturally speaking, the ontology is a part of the world and is accessible through the world object. Objects created in a world will only appear in the ontology of that world.

Extending the Ontology

Feel free to extend the ontology with new classes and relations as you go! These will be then reviewed in the pull requests so don't be shy!

If you are looking to integrate PyCRAP with some other ontology like [Soma](#), you can do so by denoting the class equivalences in the PyCRAP ontology. Details on this are found in the [owlready2 properties documentation](#).

Currently, only objects spawned during runtime are tracked in the A-Box of PyCRAP.

The roadmap for a full integration is as follows: - Map the entire belief state in the A-Box, including - Relevant links from the URDF - Robot descriptions - Find a way to get action descriptions from ontological statements - Use the ontology to guide the robot in its decision making

How to create Knowledge Source

This notebook will detail what a knowledge source does, how it works and how you can create your own.

A knowledge source is part of the wider knowledge system in PyCRAM as explained [here](#). The purpose of a knowledge source is to provide an interface to an external knowledge and reasoning system.

A knowledge source essentially consists of two parts, management methods which take care of connecting to the knowledge system as well as registering the knowledge source with the knowledge engine and the implementation of the respective reasoning queries which this knowledge source is able to process.

In this example we will walk through the process of creating a simple knowledge source and all steps involved in this process.

Creating the Knowledge Source structure

We will start by creating the general structure of the Knowledge Source as well as the management methods. To do this you have to create a new class which inherits from the KnowledgeSource class.

```
from pycram.knowledge.knowledge_source import KnowledgeSource

class ExampleKnowledge(KnowledgeSource):

    def __init__(self):
        super().__init__(name="example", priority=0)
        self.parameter = {}
```

```

def is_available(self) -> bool:
    return True

def is_connected(self) -> bool:
    return True

def connect(self):
    pass

def clear_state(self):
    self.parameter = {}

```

What we did in the code above was creating a class which inherits from the `KnowledgeSource` base class, in the constructor of this new class we initialize the base class with the name of the new Knowledge Source as well as a priority. The priority is used to determine the order of all Knowledge Sources, in case two Knowledge Sources provide the same reasoning queries the one with the higher priority is used.

Furthermore, we define a number of methods that manage the connection to the knowledge system namely the methods `is_available`, `is_connected` and `connect`. The first two methods just return a bool which states if the knowledge system is available and connected to this Knowledge Source. The last method is used to create a connection to the knowledge system. Since this is an example and we are not connecting to an external knowledge system the methods are fairly trivial.

The last method we defined is `clear_state`, this is used to clear any state the knowledge source itself might hold either of the knowledge system or of this Knowledge Source class itself.

Managing the resolvable Properties

Properties serve two purposes in the management of knowledge in PyCRAM, the first is to define semantic properties of parameter of action designator. The second is to specify which properties or knowledge queries a Knowledge Source can answer.

To define which properties a Knowledge Source can handle we simply use the respective properties as mix-in for the class definition. With this let's take another look at our Knowledge Source with the handling of two properties.

```

from pycram.knowledge.knowledge_source import KnowledgeSource
from pycram.datastructures.property import ReachableProperty,
SpaceIsFreeProperty
from pycram.datastructures.world import World
from pycram.datastructures.pose import Pose
from pycram.datastructures.dataclasses import ReasoningResult
from pycram.costmaps import OccupancyCostmap
from pycram.ros import loginfo
import numpy as np

```

```

class ExampleKnowledge(KnowledgeSource, ReachableProperty,
SpaceIsFreeProperty):

    def __init__(self):
        super().__init__(name="example", priority=0)
        self.parameter = {}

    def is_available(self) -> bool:
        return True

    def is_connected(self) -> bool:
        return True

    def connect(self):
        pass

    def clear_state(self):
        self.parameter = {}

    def reachable(self, pose: Pose) -> ReasoningResult:
        loginfo(f"Checking reachability for pose {pose}")
        robot_pose = World.robot.pose
        distance = pose.dist(robot_pose)
        return ReasoningResult(distance < 0.6)

    def space_is_free(self, pose: Pose) -> ReasoningResult:
        loginfo(f"Checking if the space is free around {pose}")
        om = OccupancyCostmap(0.2, False, 100, 0.02, pose)
        return ReasoningResult(np.sum(om.map) == 6561)

```

Now we extend our Knowledge Source with the capability to handle two properties, Reachable and SpaceIsFree. As you can see all we needed to do for this is to use the respective properties as mix-ins besides the Knowledge Source as well as implement the method for each property which does the actual reasoning.

In this case the reasoning is kept fairly simple, since this is not the objective of this example. Reachable just checks if a pose is within 60 centimeters of the robot while SpaceIsFree checks if a 2x2 meter square around the given pose has no obstacles.

The methods doing the reasoning have to return a `ReasoningResult` instance, which contains a bool stating if the reasoning succeeded or failed as well as additional parameters which might be inferred during reasoning. The additional parameters are stated as key-value pairs in a dictionary.

Testing the Knowledge Source

Since we now have a Knowledge Source which also implements two properties we can check if the Knowledge Source is used to resolve the correct properties.

For this test we need a world as well as a robot.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import WorldMode, ObjectType
from pycram.knowledge.knowledge_engine import KnowledgeEngine
from pycram.datastructures.pose import Pose
from pycram.datastructures.property import ReachableProperty,
SpaceIsFreeProperty
from pycrap.ontologies import Robot

world = BulletWorld(WorldMode.GUI)
pr2 = Object("pr2", Robot, "pr2.urdf")

target_pose = Pose([0.3, 0, 0.2])
property = ReachableProperty(target_pose) &
SpaceIsFreeProperty(target_pose)

ke = KnowledgeEngine()
resolved_property = ke.resolve_properties(property)

print(f"Result of the property: {resolved_property()}")
```

As you can see we created a `ReachableProperty` as well as a `SpaceIsFreeProperty` and resolved them. For more details on how properties and their resolution work please refer to the properties example.

Afterwards, we execute the properties, here we can see the logging infos from our Knowledge Source as well as the confirmation that the implementation for both properties worked correctly.

```
world.exit()
```

Property

Properties are the part of knowledge which is most prominent to the user. In this example we will go over what properties are, how they are defined and how they integrate into the bigger knowledge system.

Properties are used in PyCRAM to annotate parameter of action designator with semantic properties. Properties are only proxy objects which do not implement the actual functionality of the property, this is done in the knowledge Source which can handle this property. For more information how Knowledge Sources work and how they interact with properties please look at the `knowledge_source` example.

In this example we will define an example property and use this as a running example to walk through all steps involved in creating a new property.

Creating a new Property

To create a new property in PyCRAM we simply need to create a new class which inherits from the `Property` class.

```
from pycram.datastructures.property import Property
from pycram.datastructures.pose import Pose
from pycram.datastructures.dataclasses import ReasoningResult
from pycram.failures import ReasoningError
from abc import abstractmethod

class ExampleProperty(Property):

    property_exception = ReasoningError

    def __init__(self, pose: Pose):
        super().__init__()
        self.pose = pose

    @abstractmethod
    def example(self, pose: Pose) -> ReasoningResult:
        raise NotImplementedError
```

And with that we created a new property. While the process of creating a new property is simple and straight forward there are still a few things a user has to adhere to that should be kept in mind.

- Only **one** method, besides the constructor, can be defined otherwise the resolution will fail.
- A `property_exception` has to be defined, which is raised in case the reasoning about the property is not
- successful
- The reasoning method of the property can have an arbitrary amount of parameter, however, for each of these parameter
- an instance variable with the same name has to be present.
 - This means for this example: when the `example` method is called later on, while evaluating all properties,
 - it will be called with the value of `self.pose` as parameter.

Now we can use this property to annotate a variable.

```
example_property = ExampleProperty(Pose([1, 1, 1]))
```

As already mentioned do properties not implement the actual reasoning functionality and are just proxy objects to annotate parameter of action designator. The reasoning functionality is implemented in a Knowledge Source, so to test our new property we need to create a Knowledge

Source which can handle the `ExampleProperty`. For detailed information how a Knowledge Source is created please refer to the respective example.

A Knowledge Source for our Property

```
from pycram.knowledge.knowledge_source import KnowledgeSource
from pycram.datastructures.pose import Pose
from pycram.datastructures.dataclasses import ReasoningResult

class ExampleKnowledge(KnowledgeSource, ExampleProperty):

    def __init__(self):
        super().__init__(name="example", priority=0)
        self.parameter = {}

    def is_available(self) -> bool:
        return True

    def is_connected(self) -> bool:
        return True

    def connect(self):
        pass

    def clear_state(self):
        self.parameter = {}

    def example(self, pose: Pose) -> ReasoningResult:
        null_pose = Pose([1, 1, 1])
        distance = null_pose.dist(pose)
        return ReasoningResult(distance == 0.0)
```

Since we now have a Knowledge Source that can handle our `ExampleProperty` we can resolve and evaluate the property. The reasoning implementation just checks if the distance between the given pose and a pose at [1, 1, 1] is zero, meaning they point at the same point.

```
from pycram.knowledge.knowledge_engine import KnowledgeEngine

ke = KnowledgeEngine()
resolved_property = ke.resolve_properties(example_property)

print(f"Result of the property: {resolved_property()}")
```

Graph of Convex Sets

This notebook walks through the way PyCRAM navigates spaces that are challenging to navigate. Usually, navigation is done in spaces that are convex, meaning that every point is reachable via a straight line without collision. Unfortunately, the real world is not like this.

PyCRAM internally represents the objects in the world using some implementation of a scene description format. These formats include collision information for every object. The collision information is often approximated using a set of boxes.

These collision boxes are converted to their algebraic representation using the [random-events](#) package. This allows the free space to be formulated as the complement of the belief state collision boxes. The complement itself is a finite collection of (possible infinitely big) boxes that do not intersect. These boxes, however, have surfaces that are adjacent. Representing this adjacency is done using a Graph of Convex Sets (GCS) where every node is a box, and every edge means that these boxes are adjacent. Navigating the free space is then possible using path finding algorithms on the graph.

You can read more about GCS [here](#).

Let's get hands on! First, we need to create an object that makes navigation non-trivial.

```
from pycram.world_concepts.world_object import Object
from pycram.datastructures.enums import WorldMode
from pycrap.ontologies import PhysicalObject

from pycram.worlds.bullet_world import BulletWorld
from pycram.object_descriptors.generic import ObjectDescription as
GenericObjectDescription
from pycram.ros_utils.viz_marker_publisher import VizMarkerPublisher

world = BulletWorld(mode=WorldMode.DIRECT)
viz_marker_publisher = VizMarkerPublisher()
obstacle = Object("obstacle", concept=PhysicalObject,
description=GenericObjectDescription("obstacle", [0,0,0.5], [0.5, 0.5,
0.5]))
-----
-
ModuleNotFoundError                                Traceback (most recent call
last)
Cell In[1], line 1
----> 1 from pycram.world_concepts.world_object import Object
      2 from pycram.datastructures.enums import WorldMode
      3 from pycrap.ontologies import PhysicalObject

ModuleNotFoundError: No module named 'pycram'
```

Next, we create a connectivity graph of the space so we can solve navigation problems. To visualize the result in a better way, we limit the search space to a finite set around the box. Furthermore, we constraint the robot to be unable to fly by constraining the z-axis. Otherwise, he would get the idea to go over the box, which is not a good idea.

```
from random_events.interval import SimpleInterval
from pycram.graph_of_convex_sets import GraphOfConvexSets
from pycram.datastructures.dataclasses import BoundingBox

search_space = BoundingBox(min_x=-1, max_x=1,
                           min_y=-1, max_y=1,
                           min_z=0.1, max_z=0.2)
gcs = GraphOfConvexSets.free_space_from_world(world,
search_space=search_space)
```

Let's have a look at the free space constructed. We can see that it is a rectangular catwalk around the obstacle.

```
import plotly
plotly.offline.init_notebook_mode()
import plotly.graph_objects as go

fig = go.Figure(gcs.plot_free_space())
fig.show()
```

Looking at the connectivity graph, we can see that it is still possible to go from one side of the box to the other, just not directly. Intuitively, we can see that we just have to go around the obstacle.

```
import matplotlib.pyplot as plt
import networkx as nx
nx.draw(gcs, with_labels=True, font_size=8)
```

Let's use graph theory to find a path!

```
from pycram.datastructures.pose import Pose

start = Pose([-0.75, 0, 0.15])
goal = Pose([0.75, 0, 0.15])
path = gcs.path_from_to(start, goal)
print("A potential path is", [(point.position.x, point.position.y) for
point in path])
```

This minimal example demonstrates a concept that can be applied to the entire belief state of the robot. Let's load a more complex environment and look at the connectivity of it.

```
from pycrap.ontologies import Kitchen

kitchen = Object("kitchen", Kitchen, "kitchen.urdf")
```

```
search_space = BoundingBox(min_x=-2, max_x=2,
                           min_y=-2, max_y=2,
                           min_z=0., max_z=2)
gcs = GraphOfConvexSets.free_space_from_world(world,
search_space=search_space)
```

We can now see the algebraic representation of the occupied and free space. The free space is the complement of the occupied space.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2, specs=[[{'type': 'surface'}, {'type':
'surface'}]], subplot_titles=["Occupied Space", "Free Space"])

occupied_traces = gcs.plot_occupied_space()
fig.add_traces(occupied_traces, rows=[1 for _ in occupied_traces], cols=[1
for _ in occupied_traces])
free_traces = gcs.plot_free_space()
fig.add_traces(free_traces, rows=[1 for _ in free_traces], cols=[2 for _
in free_traces])
fig.show()
```

Now let's look at the connectivity of the entire world!

```
import networkx as nx
nx.draw(gcs, node_size=10)
```

We can see that all spaces are somehow reachable from everywhere besides one isolated region! Amazing! This allows the accessing of locations using a sequence of local problems put together in an overarching trajectory! Finally, let's find a way from here to there:

```
start = Pose([-0.75, 0, 0.15])
goal = Pose([0.75, 0, 0.15])
path = gcs.path_from_to(start, goal)
print("A potential path is", [(point.position.x, point.position.y,
point.position.z) for point in path])
```

Known limitations and potential improvements are:

- The connectivity graph currently calculates its edges by using an approximation to adjacent surfaces. This can be improved by an exact calculation.
- The path is generated through the center points of the connection boxes. This is perhaps not optimal
- The path is chosen by taking the shortest (meaning the least amount of edges) path. This is not necessarily the best path